

TITLE OF THE INVENTION

Universal Rail-to-ISO 20022 Interoperability Bridge (URIB)

A Canonical Event Translation and Settlement Framework for Any Blockchain or Financial Rail

ABSTRACT

A universal interoperability system enabling any distributed ledger, blockchain, or financial rail to communicate using ISO 20022 financial messaging standards. The system includes inbound rail adapters, a canonical event model, an ISO 20022 mapping engine, a policy and compliance engine, outbound rail adapters, and a cross-rail settlement coordinator. The invention allows Bitcoin, XRP, Stellar, EVM chains, private ledgers, and banking systems to interoperate through a unified, deterministic, audit-grade messaging fabric.

BACKGROUND OF THE INVENTION

Current blockchain ecosystems operate in isolation, with incompatible transaction formats, settlement semantics, and messaging structures. Financial institutions rely on ISO 20022, while blockchains use proprietary formats. Existing bridges are rail-specific, brittle, and lack auditability.

There is no universal system that:

- normalizes events from ANY rail
- maps them to ISO 20022
- applies compliance and routing
- executes settlement on ANY destination rail
- reconstructs lifecycle events deterministically

SUMMARY OF THE INVENTION

The invention provides a **Universal Rail-to-ISO 20022 Interoperability Bridge (URIB)** that enables any blockchain, distributed ledger, or financial rail to communicate using ISO 20022 financial messaging standards. The system introduces a **Canonical Event Model (CEM)** that normalizes heterogeneous transaction formats into a unified schema. A **Mapping Engine** converts CEM instances into ISO 20022 messages such as pacs.008, pain.001, seev.031, colr.001, and fxtr.001. A **Policy and Compliance Engine** applies jurisdictional, regulatory, and covenant-based constraints. A **Rail Profile Object** defines how each rail is read, written, and mapped. A **Settlement Coordinator** executes native settlement instructions on destination rails. A **Lifecycle Reconstruction Engine** provides deterministic replay of cross-rail events for audit and compliance purposes. The invention enables universal interoperability across blockchains, banking systems, CBDCs, and enterprise ledgers using ISO 20022 as the common settlement language.

Best Mode for Implementing the Universal Rail-to-ISO Bridge

The best mode currently contemplated for implementing the Universal Rail-to-ISO Bridge (URIB) uses the following architecture:

1. **Canonical Event Model (CEM v1)**
Implemented as a JSON-serializable schema containing fields for event_type, asset_type, asset_identity, amount, sender, receiver, metadata_hash, timestamp, rail_origin, rail_destination, compliance_flags, and settlement_mode.
2. **Inbound Rail Adapters**
Implemented as modular components written in a high-level language such as Python or Go.
 - Bitcoin adapter uses RPC or Electrum-style APIs to extract UTXO data.
 - XRP adapter uses WebSocket or JSON-RPC to extract ledger objects.
 - EVM adapter uses Web3 interfaces to extract logs and state.
 - Stellar adapter uses Horizon API.
 - Bank rails use REST or MQ-based ISO 20022 endpoints.
3. **ISO 20022 Mapping Engine**
Implemented as a rule-based transformer that maps CEM fields into XML or JSON ISO 20022 structures.
Mapping tables are stored as versioned JSON files.
4. **Policy & Compliance Engine**
Implemented as a rule evaluator that checks compliance_flags, jurisdiction, and covenant_state.
Rules are expressed as JSON logic or a domain-specific rule language.
5. **Outbound Rail Adapters**
Implemented similarly to inbound adapters but in reverse:
 - Bitcoin: constructs raw transactions and signs them.
 - XRP: constructs Payment, TrustSet, or NFToken operations.
 - EVM: constructs contract calls or token transfers.
 - Stellar: constructs operations via Horizon.
 - Bank rails: sends ISO 20022 messages via API.
6. **Settlement Coordinator**
Implements atomicity using a two-phase commit or hash-time-locked mechanism depending on rail capabilities.
7. **Lifecycle Reconstruction Engine**
Stores CEM logs, ISO logs, and settlement logs in append-only storage.
Replay engine reconstructs events by re-executing transformations in order.

This configuration represents the best mode known at the time of filing for achieving universal ISO 20022 interoperability across heterogeneous financial rails.

PROPOSED FIGURE DESCRIPTIONS

FIG. 1 — System Architecture Overview

FIG. 1 illustrates the Universal Rail-to-ISO Bridge architecture, including inbound rail adapters, the Canonical Event Model, the ISO 20022 Mapping Engine, the Policy and Compliance Engine, outbound rail adapters, and the Settlement Coordinator.

FIG. 2 — Canonical Event Model (CEM) Structure

FIG. 2 shows the CEM schema, including fields for event_type, asset_type, asset_identity, amount, sender, receiver, metadata_hash, timestamp, rail_origin, rail_destination, compliance_flags, and settlement_mode.

FIG. 3 — Inbound Rail Adapter Flow

FIG. 3 illustrates the process of receiving a native transaction from a source rail, extracting rail-specific attributes, and generating a CEM instance.

FIG. 4 — ISO 20022 Mapping Flow

FIG. 4 shows the transformation of a CEM instance into an ISO 20022 message using mapping tables and schema selection logic.

FIG. 5 — Outbound Rail Adapter Flow

FIG. 5 illustrates the conversion of an ISO 20022 message into a native settlement instruction for a destination rail.

FIG. 6 — Cross-Rail Settlement Sequence

FIG. 6 shows the sequence of operations for executing settlement across two or more rails, including canonicalization, ISO mapping, compliance evaluation, and settlement execution.

FIG. 7 — Lifecycle Reconstruction Engine

FIG. 7 illustrates the logging of CEM events, ISO messages, and settlement confirmations, and the replay engine used for deterministic reconstruction.

FIG. 8 — Rail Profile Object Structure

FIG. 8 shows the components of a Rail Profile, including `read_method`, `write_method`, `event_extractor`, `signature_verifier`, `settlement_method`, and `mapping_rules`.

FIG. 9 — Multi-Rail Routing Example

FIG. 9 illustrates routing from Bitcoin → ISO → XRP → ISO → Bank rail using the universal bridge.

FIG. 10 — RWA and SST-10K Integration

FIG. 10 shows how SST-10K or RWA-bound assets are represented in the CEM and mapped into ISO 20022 securities and collateral messages.

SYSTEM OVERVIEW

The Universal Rail-to-ISO 20022 Interoperability Bridge (URIB) is a modular, extensible architecture designed to normalize, translate, and settle financial events across heterogeneous blockchains, distributed ledgers, and traditional financial rails. The system operates as a multi-stage processing pipeline in which native events from any source rail are extracted, canonicalized, mapped into ISO 20022 financial messages, evaluated for compliance, and executed as settlement instructions on any destination rail. The architecture is rail-agnostic and message-agnostic, enabling interoperability between systems that use fundamentally different transaction models, data structures, and settlement semantics.

At the entry point of the system, inbound rail adapters interface directly with source networks such as Bitcoin, XRP Ledger, Stellar, EVM chains, Hyperledger, CBDC platforms, and banking APIs. These adapters extract rail-specific attributes and convert them into a unified Canonical Event Model (CEM), which represents the essential semantics of the event in a deterministic, rail-independent format. The CEM serves as the central data structure that drives all downstream processing.

The ISO 20022 Mapping Engine transforms CEM instances into ISO 20022 messages by applying deterministic field-to-field mappings defined in versioned mapping tables. These messages are then evaluated by the Policy and Compliance Engine, which applies jurisdictional rules, AML/KYC indicators, covenant constraints, and routing policies to determine whether and how the event may proceed.

Outbound rail adapters convert approved ISO 20022 messages into native settlement instructions for the destination rail, enabling cross-rail execution across both blockchain and traditional financial systems. A Settlement Coordinator ensures atomicity, ordering, rollback, and multi-rail consistency during execution. All stages of the process—including canonicalization, mapping, compliance evaluation, and settlement—are recorded by the Lifecycle Reconstruction Engine, which provides deterministic replay and audit-grade traceability across the entire event lifecycle.

Together, these components form a unified interoperability framework that enables any blockchain or financial rail to communicate and settle through ISO 20022, establishing a universal, deterministic, and extensible foundation for cross-rail financial messaging and settlement.

DEFINITIONS

For purposes of this specification, the following terms shall have the meanings set forth below. These definitions are provided to ensure clarity, consistency, and unambiguous interpretation of the system architecture, data structures, and operational flows described herein.

“Rail” or “Financial Rail”

A blockchain, distributed ledger, payment network, banking system, CBDC platform, or any other system capable of originating, transmitting, or settling financial transactions. Examples include Bitcoin, XRP Ledger, Stellar, EVM chains, Hyperledger, ACH, SWIFT, and ISO 20022-based banking APIs.

“Native Transaction Event”

A transaction, operation, ledger entry, or state transition emitted by a source rail in its original, rail-specific format. Examples include Bitcoin UTXO spends, XRP Payment operations, Stellar ManageData entries, EVM contract logs, or ISO 20022 pacs/pain messages.

“Inbound Rail Adapter”

A software component configured to interface with a specific rail, extract native transaction attributes, and convert them into a normalized intermediate representation suitable for canonicalization.

“Canonical Event Model (CEM)”

A rail-agnostic, deterministic data structure representing the essential semantics of a financial event. The CEM abstracts away rail-specific formats and provides a unified schema for ISO 20022 mapping, compliance evaluation, and settlement execution.

“Canonicalization”

The process of transforming a native transaction event into a Canonical Event Model instance by normalizing rail-specific attributes into standardized fields.

“ISO 20022 Mapping Engine”

A rule-based transformation component that converts CEM instances into ISO 20022 messages using deterministic field-to-field mapping tables.

“ISO 20022 Message”

A structured financial message conforming to ISO 20022 standards, including but not limited to pacs.008, pain.001, seev.031, colr.001, fxtr.001, and camt.054.

“Mapping Table”

A versioned set of transformation rules defining how CEM fields correspond to ISO 20022 message elements.

“Policy & Compliance Engine”

A rule evaluation component that applies jurisdictional constraints, AML/KYC indicators, covenant restrictions, and routing policies to determine whether a mapped ISO 20022 message is eligible for settlement.

“Covenant State”

A set of constraints, conditions, or rules associated with an asset or transaction, including no-split rules, no-merge rules, time locks, oracle conditions, or RWA binding requirements.

“Outbound Rail Adapter”

A software component that converts ISO 2002 messages into native settlement instructions for a destination rail.

“Settlement Instruction”

A rail-specific command or transaction that executes the financial event on the destination rail, such as a Bitcoin transaction, XRP Payment, Stellar operation, EVM contract call, or ISO 20022 pacs/pain submission.

“Settlement Coordinator”

A component responsible for ensuring atomicity, ordering, rollback, and multi-rail consistency during cross-rail settlement execution.

“Atomic Settlement”

A settlement process in which all required actions across multiple rails succeed or none do, ensuring no partial or inconsistent state.

“Lifecycle Reconstruction Engine”

A component that records and replays all stages of cross-rail processing—including canonicalization, ISO mapping, compliance evaluation, and settlement execution—to provide deterministic auditability.

“Deterministic Replay”

The ability to reconstruct the exact sequence of transformations and settlement actions for any event using logged data, ensuring audit-grade traceability.

“Rail Profile Object”

A modular configuration object defining how a specific rail is read, written, validated, and mapped, including `read_method`, `write_method`, `event_extractor`, `signature_verifier`, `settlement_method`, and `mapping_rules`.

“Metadata Hash”

A cryptographic hash representing inscriptions, covenant state, RWA metadata, or other auxiliary data associated with a transaction or asset.

“Proof Data”

Cryptographic evidence supporting the authenticity or validity of a transaction or state, including SPV proofs, Merkle paths, or zero-knowledge proofs.

DETAILED DESCRIPTION

1. Inbound Rail Adapters

Inbound Rail Adapters are modular software components configured to interface with heterogeneous financial rails and extract native transaction attributes in their original formats. Each adapter implements a rail-specific extraction pipeline but exposes a uniform output structure for canonicalization.

1.1 Bitcoin Adapter

The Bitcoin adapter retrieves transaction data from a Bitcoin node or indexer using RPC, Electrum, or REST interfaces. It extracts:

- UTXO inputs and outputs
- Taproot script paths
- Ordinal inscriptions
- SST-10K structured metadata
- Block height, timestamp, and Merkle proofs

The adapter normalizes these into abstract fields such as sender, receiver, asset_identity, and metadata_hash.

1.2 XRP Ledger Adapter

The XRP adapter connects via WebSocket or JSON-RPC to retrieve:

- Payment operations
- Trustline modifications
- Issued asset transfers
- NFToken operations
- Ledger index, sequence, and signature bundles

It extracts sender, receiver, asset_type, amount, and ledger metadata.

1.3 Stellar Adapter

The Stellar adapter uses Horizon APIs to extract:

- Payment operations
- ManageData entries
- Trustline operations
- Memo fields
- Sequence numbers and signatures

1.4 EVM Adapter

The EVM adapter uses Web3 interfaces to extract:

- Contract logs
- ERC-20/721/1155 transfers
- Smart contract state transitions
- Gas usage and block metadata

1.5 Hyperledger Adapter

The Hyperledger adapter extracts:

- Chaincode events
- Private data collections
- Identity certificates
- Transaction envelopes

1.6 CBDC Rail Adapter

For CBDC systems, the adapter retrieves:

- Token movements
- Identity-bound metadata
- Jurisdictional flags
- Central bank signatures

1.7 Banking API Adapter

For traditional rails, the adapter ingests:

- ISO 20022 messages
- ACH/NACHA entries
- SWIFT MT/MX messages
- Proprietary bank APIs

1.8 Normalization

All adapters convert extracted attributes into a unified intermediate representation, which is then transformed into a Canonical Event Model (CEM) instance.

2. Canonical Event Model (CEM)

The Canonical Event Model is a rail-agnostic schema that represents the essential semantics of any financial event. It abstracts away rail-specific structures and provides a deterministic, machine-readable representation suitable for ISO 20022 mapping.

```
CanonicalEventModel {  
    event_type  
    asset_type  
    asset_class  
    asset_identity  
    amount  
    unit  
    sender  
    receiver  
    metadata_hash  
    metadata_uri  
    rail_origin  
    rail_destination  
    compliance_flags  
    jurisdiction  
    covenant_state  
    timestamp  
    expiry_time  
    proof_type  
    proof_data  
    settlement_mode  
    settlement_priority  
}
```

2.1 Universal Canonical Mapping Layer (UCML)

(paste this directly into your provisional)

The invention introduces a Universal Canonical Mapping Layer (UCML) that defines deterministic, reversible transformation rules between heterogeneous financial rails and the Canonical Event Model (CEM). The UCML provides:

- Rail → Canonical mapping tables
- Canonical → Rail mapping tables
- Field-level mappings
- Operation-level mappings
- Compliance-level mappings
- Replay-level mappings
- Atomicity-level mappings
- Settlement-level mappings
- Identity-level mappings
- Metadata-level mappings

Each rail implements a reversible mapping suite that ensures deterministic translation of payment primitives, asset descriptors, compliance metadata, identity structures, settlement coordinates, replay coordinates, liquidity structures, escrow primitives, and atomicity primitives. The UCML ensures that all rails—regardless of structure—can be represented in a unified canonical format suitable for ISO 20022 mapping, cross-rail settlement, and deterministic lifecycle reconstruction.

2.2 Universal Cross-Rail Equivalence Matrix (UCEM)

(paste this directly into your provisional)

The invention introduces a Universal Cross-Rail Equivalence Matrix (UCEM) that defines deterministic equivalence relationships across heterogeneous financial rails. The UCEM provides equivalence mappings for:

- Payment primitives
- Routing primitives
- FX and AMM behavior
- Compliance semantics
- Settlement semantics
- Atomicity primitives
- Replay and lifecycle semantics
- Identity models
- Trustline and credit models
- Liquidity and pool structures

The UCEM enables the system to determine that a Stellar PaymentOp, an XRP Payment, an Ethereum ERC-20 transfer, a CBDC token movement, and a banking pacs.008 instruction are semantically equivalent under the canonical model. The equivalence matrix enables deterministic multi-rail routing, deterministic fallback selection, deterministic replay, deterministic compliance harmonization, and deterministic settlement ordering.

2.3 Field Semantics

- `event_type`: transfer, mint, burn, lock, unlock, trade, pledge
- `asset_type`: BTC, XRP, XLM, ERC 20, RWA token, SST 10K
- `asset_class`: payment, security, collateral, commodity
- `asset_identity`: Genesis Sat ID, token hash, contract address
- `metadata_hash`: hash of inscriptions, covenant state, RWA metadata
- `compliance_flags`: AML, jurisdiction, covenant, asset restrictions
- `proof_data`: SPV proofs, Merkle paths, ZK proofs

The CEM ensures deterministic mapping across all rails.

3. ISO 20022 Mapping Engine

The ISO 20022 Mapping Engine transforms CEM instances into ISO 20022 messages using deterministic mapping tables. The engine selects the appropriate ISO schema based on `event_type` and `asset_class`.

3.1 Supported ISO Messages

- **pacs.008** — cross-border payments
- **pain.001** — customer credit transfers
- **seev.031** — securities settlement instructions
- **colr.001** — collateral instructions
- **fxtr.001** — FX trade instructions
- **camt.054** — bank account notifications

3.2 Mapping Logic

The engine applies field-to-field transformations such as:

- `CEM.amount` → `ISO.InstructedAmount`
- `CEM.asset_identity` → `ISO.FinancialInstrumentID`
- `CEM.metadata_hash` → `ISO.SupplementaryData`
- `CEM.compliance_flags` → `ISO.RegulatoryReporting`

Mapping tables are versioned and stored as JSON.

3.3 Output

The engine produces ISO 20022 messages in XML or JSON format, ready for compliance evaluation or outbound settlement.

4. Policy & Compliance Engine

The Policy & Compliance Engine evaluates regulatory, jurisdictional, and covenant-based constraints before settlement.

4.1 Jurisdiction Rules

Checks:

- sender/receiver country
- CBDC jurisdiction
- asset-class restrictions

4.2 AML/KYC Flags

Evaluates:

- risk scores
- transaction patterns
- identity metadata

4.3 Covenant Constraints

Enforces:

- no-split rules
- no-merge rules
- time-locked restrictions
- RWA binding conditions

4.4 Routing Policies

Determines:

- allowed rails
- fallback rails
- multi-hop routing

5. Outbound Rail Adapters

Outbound adapters convert ISO 20022 messages into native settlement instructions.

5.1 Bitcoin

Constructs:

- raw transactions
- Taproot scripts
- ordinal inscriptions
- covenant-locked outputs

5.2 XRP Ledger

Constructs:

- Payment
- TrustSet
- NFToken operations

5.3 Stellar

Constructs:

- Payment
- ManageData
- ChangeTrust

5.4 EVM

Constructs:

- ERC-20 transfers
- contract calls
- multi-sig bundles

5.5 Banking Rails

Submits:

- ISO 20022 pacs/pain messages
- ACH entries
- SWIFT MX messages

6. Settlement Coordinator

The Settlement Coordinator ensures deterministic cross-rail execution.

6.1 Atomicity

Uses:

- two-phase commit
- hash-time-locked contracts
- conditional ISO instructions

6.2 Ordering

Ensures:

- canonical ordering
- replay-safe sequencing

6.3 Rollback

If a destination rail fails, the coordinator:

- cancels pending instructions
- reverts canonical state
- logs failure events

6.4 Multi-Rail Consistency

Ensures all rails reflect the same final state.

7. Lifecycle Reconstruction Engine

The Lifecycle Reconstruction Engine provides full auditability.

7.1 Logs

Stores:

- CEM logs
- ISO 20022 logs
- settlement confirmations

7.2 Deterministic Replay

Reconstructs:

- inbound extraction
- canonicalization
- ISO mapping
- settlement execution

7.3 Audit Output

Produces:

- compliance reports
- cross-rail timelines
- cryptographic proofs

ISO 20022 MAPPING TABLES

These tables define how your **Canonical Event Model (CEM)** fields map into **ISO 20022 message structures**.

TABLE A — Core Payment Mapping (CEM → ISO 20022 pacs.008 / pain.001)

CEM Field	ISO 20022 Concept	Mapping Logic
event_type	PaymentInstruction	If event_type = “transfer”, generate payment instruction
asset_type	InstructedAmount/Currency	Map asset_type to currency code or token identifier
amount	InstructedAmount/Value	Direct numeric mapping
sender	Debtor/Account	Map sender rail address → debtor account structure
receiver	Creditor/Account	Map receiver rail address → creditor account structure
metadata_hash	RemittanceInformation	Insert hash as structured remittance data
timestamp	InterbankSettlementDate	Convert block/ledger timestamp to settlement date
rail_origin	DebtorAgent	Identify originating rail as sending agent
rail_destination	CreditorAgent	Identify destination rail as receiving agent
compliance_flags	RegulatoryReporting	Insert compliance flags into regulatory block

TABLE B — Securities Mapping (CEM → ISO 20022 seev.031 / seev.032)

CEM Field	ISO 20022 Concept	Mapping Logic
asset_type	FinancialInstrumentID	Map SST-10K, RWA, NFT, token ID → instrument ID
asset_identity	SecurityIdentification	Map Genesis Sat ID or token hash
amount	Quantity	Map satoshi count or token quantity
sender	DeliveringParty	Originating rail address
receiver	ReceivingParty	Destination rail address
metadata_hash	SupplementaryData	Insert as instrument metadata
compliance_flags	SettlementParameters	Apply restrictions or conditions
timestamp	TradeDate	Map ledger timestamp to trade date

TABLE C — Collateral Mapping (CEM → ISO 20022 colr.001 / colr.002)

CEM Field	ISO 20022 Concept Mapping Logic	
asset_type	CollateralType	Map SST-10K, RWA, tokenized asset
asset_identity	CollateralID	Map Genesis Sat ID or token hash
amount	CollateralValue	Map quantity or valuation
sender	Pledgor	Entity pledging collateral
receiver	SecuredParty	Entity receiving collateral
metadata_hash	CollateralAttributes	Insert metadata hash
compliance_flags	Eligibility	Apply covenant or regulatory constraints

TABLE D — FX Mapping (CEM → ISO 20022 fxtr.001 / fxtr.003)

CEM Field	ISO 20022 Concept	Mapping Logic
asset_type	TradedCurrency	Map token/asset to currency code
amount	TradedAmount	Map numeric value
rail_origin	TradingParty1	Originating rail
rail_destination	TradingParty2	Destination rail
metadata_hash	AdditionalInformation	Insert structured metadata
compliance_flags	RegulatoryReporting	Insert compliance flags

TABLE E — Audit & Lifecycle Mapping (CEM → ISO 20022 camt.053 / camt.054 / acmt.022)

CEM Field	ISO 20022 Concept Mapping Logic	
event_type	EntryType	Map transfer, mint, burn, lock, unlock
amount	EntryAmount	Map numeric value
sender	AccountOwner	Map originating address
receiver	Counterparty	Map destination address
timestamp	BookingDate	Map ledger timestamp
metadata_hash	EntryDetails	Insert metadata hash
rail_origin	AccountServicer	Identify originating rail
rail_destination	CounterpartyAgent	Identify destination rail

TABLE F — Covenant & Policy Mapping (CEM → ISO 20022 seev.044 / admi.003)

CEM Field	ISO 20022 Concept Mapping Logic	
compliance_flags	ConstraintIndicator	Map covenant rules (no-split, no-merge, atomic)
event_type	InstructionType	Map covenant-triggered actions
metadata_hash	SupplementaryData	Insert covenant state hash
asset_identity	SecurityID	Map covenant-locked asset ID
timestamp	EffectiveDate	Map covenant activation time

IMPLEMENTATION APPENDIX — CANONICAL EVENT MODEL SPECIFICATION

1. **Canonical Event Model (CEM v1)**

Implemented as a JSON-serializable schema containing fields for `event_type`, `asset_type`, `asset_identity`, `amount`, `sender`, `receiver`, `metadata_hash`, `timestamp`, `rail_origin`, `rail_destination`, `compliance_flags`, and `settlement_mode`.

2. **Inbound Rail Adapters**

Implemented as modular components written in a high-level language such as Python or Go.

- Bitcoin adapter uses RPC or Electrum-style APIs to extract UTXO data.
- XRP adapter uses WebSocket or JSON-RPC to extract ledger objects.
- EVM adapter uses Web3 interfaces to extract logs and state.
- Stellar adapter uses Horizon API.
- Bank rails use REST or MQ-based ISO 20022 endpoints.

3. **ISO 20022 Mapping Engine**

Implemented as a rule-based transformer that maps CEM fields into XML or JSON ISO 20022 structures.

Mapping tables are stored as versioned JSON files.

4. **Policy & Compliance Engine**

Implemented as a rule evaluator that checks `compliance_flags`, jurisdiction, and `covenant_state`.

Rules are expressed as JSON logic or a domain-specific rule language.

5. **Outbound Rail Adapters**

Implemented similarly to inbound adapters but in reverse:

- Bitcoin: constructs raw transactions and signs them.
- XRP: constructs Payment, TrustSet, or NFToken operations.
- EVM: constructs contract calls or token transfers.
- Stellar: constructs operations via Horizon.
- Bank rails: sends ISO 20022 messages via API.

6. **Settlement Coordinator**

Implements atomicity using a two-phase commit or hash-time-locked mechanism depending on rail capabilities.

7. **Lifecycle Reconstruction Engine**

Stores CEM logs, ISO logs, and settlement logs in append-only storage.

Replay engine reconstructs events by re-executing transformations in order.

This configuration represents the best mode known at the time of filing for achieving universal ISO 20022 interoperability across heterogeneous financial rails.

IMPLEMENTATION APPENDIX – CANONICAL EVENT MODEL & SYSTEM COMPONENTS

This Implementation Appendix provides additional technical detail regarding the construction, operation, and interaction of the components described in the Universal Rail-to-ISO 20022 Interoperability Bridge (URIB). The following sections describe one or more preferred implementations but do not limit the scope of the invention.

1. Canonical Event Model (CEM v1)

The Canonical Event Model (CEM) is implemented as a JSON-serializable object.

In one implementation, the CEM is represented as:

```
{
  "event_type": "transfer",
  "asset_type": "BTC",
  "asset_class": "payment",
  "asset_identity": "genesis_sat_id_or_token_hash",
  "amount": 100000,
  "unit": "satoshi",
  "sender": "rail_specific_address",
  "receiver": "rail_specific_address",
  "metadata_hash": "sha256_hash",
  "metadata_uri": "optional_uri",
  "rail_origin": "bitcoin",
  "rail_destination": "xrp",
  "compliance_flags": ["aml_ok", "jurisdiction_us"],
  "jurisdiction": "US",
  "covenant_state": "no_split",
  "timestamp": 1710000000,
  "expiry_time": null,
  "proof_type": "merkle",
  "proof_data": "hex_encoded_proof",
  "settlement_mode": "atomic",
  "settlement_priority": "high"
}
```

1.1 Serialization

The CEM may be serialized as:

- JSON
- CBOR
- Protobuf
- XML

JSON is preferred for readability and interoperability.

1.2 Validation

A schema validator ensures:

- required fields are present
- field types are correct
- asset_class matches asset_type
- compliance_flags are recognized

2. Inbound Rail Adapters

Inbound Rail Adapters are implemented as modular plug-ins.

Each adapter exposes a standard interface:

extract(event) → intermediate_representation

normalize(intermediate_representation) → CEM

2.1 Bitcoin Adapter Implementation

Uses:

- getrawtransaction
- getblock
- scantxoutset
- Electrum blockchain.transaction.get

Extracts:

- inputs/outputs
- Taproot control blocks
- ordinal inscription envelopes
- SST-10K metadata
- Merkle proofs

2.2 XRP Adapter Implementation

Uses:

- ledger_data
- tx
- subscribe WebSocket streams

Extracts:

- Payment
- TrustSet
- NFTTokenMint
- NFTTokenBurn
- AMM operations

2.3 EVM Adapter Implementation

Uses:

- eth_getTransactionReceipt
- eth_getLogs
- ABI decoding

Extracts:

- ERC-20 transfers
- ERC-721/1155 events
- contract state transitions

2.4 Stellar Adapter Implementation

Uses:

- Horizon /transactions
- Horizon /operations
- Horizon /effects

Extracts:

- Payment
- ManageData
- ChangeTrust

2.5 Banking Adapter Implementation

Supports:

- ISO 20022 XML ingestion
- SWIFT MT/MX parsing
- ACH/NACHA file parsing

3. ISO 20022 Mapping Engine

The Mapping Engine is implemented as a rule-based transformer.

3.1 Mapping Table Format

Mapping tables are stored as versioned JSON:

```
{
  "version": "1.0.0",
  "cem_field": "amount",
  "iso_path": "Document.FIToFICstmrCdtTrf.InstrInf.InstdAmt.Value",
  "transform": "numeric"
}
```

3.2 Schema Selection Logic

Example:

```
if CEM.asset_class == "payment":
    use pacs.008
elif CEM.asset_class == "security":
    use seev.031
elif CEM.asset_class == "collateral":
    use colr.001
```


3.3 Output Formats

The engine can output:

- ISO 20022 XML
- ISO 20022 JSON
- hybrid JSON-XML for internal routing

4. Policy & Compliance Engine

The engine evaluates rules expressed in:

- JSON Logic
- domain-specific rule language (DSL)
- embedded Python/Go expressions

4.1 Example Rule

IF jurisdiction == "US" AND asset_class == "security"
THEN require KYC flag AND require RWA metadata

4.2 Rule Execution

Rules are executed in order:

1. jurisdiction
2. AML/KYC
3. covenant
4. routing

5. Outbound Rail Adapters

Outbound adapters expose:

translate(iso_message) → rail_instruction

submit(rail_instruction) → settlement_result

5.1 Bitcoin Implementation

Constructs:

- raw transaction hex
- Taproot witness stack
- Schnorr signatures

5.2 XRP Implementation

Constructs:

- Payment
- TrustSet
- NFTokenMint
- NFTokenBurn

5.3 EVM Implementation

Constructs:

- ABI-encoded contract calls
- ERC-20 transfers
- multi-sig bundles

5.4 Banking Implementation

Submits:

- pacs.008
- pain.001
- camt.054

via MQ, REST, or SFTP.

6. Settlement Coordinator

The Settlement Coordinator ensures deterministic cross-rail execution.

6.1 Two-Phase Commit

Phase 1: prepare

Phase 2: commit

6.2 HTLC Mode

Uses:

- hashlock
- timelock
- preimage reveal

6.3 Failure Handling

If any rail fails:

- rollback
- cancel pending instructions
- log failure event

7. Lifecycle Reconstruction Engine

The engine stores:

- CEM logs
- ISO logs
- settlement logs

7.1 Log Format

```
{  
  "cem": {...},  
  "iso": "<xml>",  
  "settlement": {...},  
  "timestamp": 1710000000  
}
```

7.2 Replay

Replay reconstructs:

1. inbound extraction
2. canonicalization
3. ISO mapping
4. compliance evaluation
5. settlement execution

7.3 Audit Output

Produces:

- chronological event timeline
- compliance report
- cryptographic proof bundle

BLOCK 1 — BITCOIN → ISO → ANY RAIL (1–50)

1. In one embodiment, the system maps Bitcoin UTXO events to ISO 20022 pacs.008 messages.
2. In one embodiment, the system converts Bitcoin ordinal transfers into ISO 20022 pain.001 messages.
3. In one embodiment, the system transforms Bitcoin Taproot spends into ISO 20022 seev.031 securities instructions.
4. In one embodiment, the system maps Bitcoin OP_RETURN metadata into ISO 20022 colr.001 collateral messages.
5. In one embodiment, the system converts Bitcoin covenant-locked outputs into ISO 20022 seev.036 settlement status messages.
6. In one embodiment, the system maps Bitcoin SST-10K movements into ISO 20022 securities settlement messages.
7. In one embodiment, the system converts Bitcoin fee events into ISO 20022 camt.054 debit notifications.
8. In one embodiment, the system maps Bitcoin multisig spends into ISO 20022 auth.001 authorization messages.
9. In one embodiment, the system converts Bitcoin RBF replacements into ISO 20022 camt.029 amendment messages.
10. In one embodiment, the system maps Bitcoin block confirmations into ISO 20022 acmt.023 status messages.
11. In one embodiment, the system maps Bitcoin P2PKH spends into ISO 20022 pacs.009 messages.
12. In one embodiment, the system maps Bitcoin P2SH spends into ISO 20022 pacs.002 messages.
13. In one embodiment, the system maps Bitcoin P2WPKH spends into ISO 20022 pacs.004 messages.
14. In one embodiment, the system maps Bitcoin P2WSH spends into ISO 20022 pacs.003 messages.
15. In one embodiment, the system maps Bitcoin OP_CHECKLOCKTIMEVERIFY events into ISO 20022 seev.032 messages.
16. In one embodiment, the system maps Bitcoin OP_CHECKSEQUENCEVERIFY events into ISO 20022 seev.033 messages.
17. In one embodiment, the system maps Bitcoin covenant violations into ISO 20022 admi.002 error messages.
18. In one embodiment, the system maps Bitcoin Taproot key-path spends into ISO 20022 pacs.008 messages.
19. In one embodiment, the system maps Bitcoin Taproot script-path spends into ISO 20022 pacs.009 messages.
20. In one embodiment, the system maps Bitcoin UTXO consolidation events into ISO 20022 camt.056 messages.
21. In one embodiment, the system maps Bitcoin UTXO fragmentation events into ISO 20022 camt.057 messages.

22. In one embodiment, the system maps Bitcoin coinbase rewards into ISO 20022 camt.053 credit notifications.
23. In one embodiment, the system maps Bitcoin difficulty adjustments into ISO 20022 admi.007 system messages.
24. In one embodiment, the system maps Bitcoin mempool acceptance events into ISO 20022 admi.004 notifications.
25. In one embodiment, the system maps Bitcoin transaction finality into ISO 20022 acmt.022 confirmation messages.
26. In one embodiment, the system maps Bitcoin RWA anchors into ISO 20022 colr.002 collateral substitution messages.
27. In one embodiment, the system maps Bitcoin covenant-locked SST-10K units into ISO 20022 seev.031 instructions.
28. In one embodiment, the system maps Bitcoin OP_RETURN hashes into ISO 20022 remt.001 remittance messages.
29. In one embodiment, the system maps Bitcoin Schnorr signatures into ISO 20022 auth.002 authentication messages.
30. In one embodiment, the system maps Bitcoin Taproot annex data into ISO 20022 remt.002 structured data messages.
31. In one embodiment, the system maps Bitcoin witness data into ISO 20022 auth.003 verification messages.
32. In one embodiment, the system maps Bitcoin covenant state transitions into ISO 20022 seev.034 messages.
33. In one embodiment, the system maps Bitcoin ordinal inscriptions into ISO 20022 remt.003 metadata messages.
34. In one embodiment, the system maps Bitcoin OP_CHECKSIG events into ISO 20022 auth.004 signature validation messages.
35. In one embodiment, the system maps Bitcoin OP_CHECKMULTISIG events into ISO 20022 auth.005 multi-party authorization messages.
36. In one embodiment, the system maps Bitcoin OP_RETURN-based receipts into ISO 20022 seev.035 receipt messages.
37. In one embodiment, the system maps Bitcoin covenant-enforced no-split events into ISO 20022 seev.037 constraint messages.
38. In one embodiment, the system maps Bitcoin covenant-enforced no-merge events into ISO 20022 seev.038 constraint messages.
39. In one embodiment, the system maps Bitcoin covenant-enforced atomic movement events into ISO 20022 seev.039 atomicity messages.
40. In one embodiment, the system maps Bitcoin covenant-enforced time locks into ISO 20022 seev.040 time-based instructions.
41. In one embodiment, the system maps Bitcoin covenant-enforced oracle locks into ISO 20022 seev.041 conditional instructions.
42. In one embodiment, the system maps Bitcoin covenant-enforced spend paths into ISO 20022 seev.042 routing messages.
43. In one embodiment, the system maps Bitcoin covenant-enforced policy failures into ISO 20022 admi.003 error messages.
44. In one embodiment, the system maps Bitcoin Taproot leaf commitments into ISO 20022 remt.004 metadata messages.

45. In one embodiment, the system maps Bitcoin Taproot Merkle paths into ISO 20022 auth.006 proof messages.
46. In one embodiment, the system maps Bitcoin UTXO aging events into ISO 20022 camt.058 lifecycle messages.
47. In one embodiment, the system maps Bitcoin RBF fee bumps into ISO 20022 camt.059 amendment messages.
48. In one embodiment, the system maps Bitcoin transaction malleability events into ISO 20022 admi.005 warnings.
49. In one embodiment, the system maps Bitcoin covenant-locked RWA units into ISO 20022 colr.003 collateral allocation messages.
50. In one embodiment, the system maps Bitcoin SST-10K receipts into ISO 20022 seev.043 securities receipt messages.

BLOCK 2 — XRP → ISO → ANY RAIL (51–100)

51. In one embodiment, the system maps XRP payments to ISO 20022 pacs.008 messages.
52. In one embodiment, the system maps XRP trustline changes to ISO 20022 seev.031 messages.
53. In one embodiment, the system maps XRP issued asset transfers to ISO 20022 seev.032 messages.
54. In one embodiment, the system maps XRP DEX trades to ISO 20022 fxtr.001 messages.
55. In one embodiment, the system maps XRP account set operations to ISO 20022 acmt.023 messages.
56. In one embodiment, the system maps XRP escrow creations to ISO 20022 colr.001 messages.
57. In one embodiment, the system maps XRP escrow releases to ISO 20022 colr.002 messages.
58. In one embodiment, the system maps XRP payment channels to ISO 20022 pacs.009 messages.
59. In one embodiment, the system maps XRP signer list updates to ISO 20022 auth.001 messages.
60. In one embodiment, the system maps XRP memo fields to ISO 20022 remt.001 messages.
61. In one embodiment, the system maps XRP NFT mints to ISO 20022 seev.033 messages.
62. In one embodiment, the system maps XRP NFT transfers to ISO 20022 seev.034 messages.
63. In one embodiment, the system maps XRP NFT burns to ISO 20022 seev.035 messages.
64. In one embodiment, the system maps XRP AMM swaps to ISO 20022 fxtr.002 messages.
65. In one embodiment, the system maps XRP AMM liquidity events to ISO 20022 seev.036 messages.
66. In one embodiment, the system maps XRP AMM LP token transfers to ISO 20022 seev.037 messages.
67. In one embodiment, the system maps XRP account deletions to ISO 20022 acmt.024 messages.
68. In one embodiment, the system maps XRP fee events to ISO 20022 camt.054 messages.
69. In one embodiment, the system maps XRP multi-sign transactions to ISO 20022 auth.002 messages.
70. In one embodiment, the system maps XRP hook executions to ISO 20022 admi.004 messages.
71. In one embodiment, the system maps XRP hook state changes to ISO 20022 remt.002 messages.
72. In one embodiment, the system maps XRP hook errors to ISO 20022 admi.003 messages.
73. In one embodiment, the system maps XRP AMM oracle price updates to ISO 20022 fxtr.003 messages.
74. In one embodiment, the system maps XRP cross-currency payments to ISO 20022 fxtr.004 messages.
75. In one embodiment, the system maps XRP pathfinding results to ISO 20022 pacs.010 messages.

76. In one embodiment, the system maps XRP payment channel claims to ISO 20022 pacs.011 messages.
77. In one embodiment, the system maps XRP payment channel creations to ISO 20022 pacs.012 messages.
78. In one embodiment, the system maps XRP payment channel closures to ISO 20022 pacs.013 messages.
79. In one embodiment, the system maps XRP signer quorum changes to ISO 20022 auth.003 messages.
80. In one embodiment, the system maps XRP account flags to ISO 20022 acmt.025 messages.
81. In one embodiment, the system maps XRP issued asset freezes to ISO 20022 seev.038 messages.
82. In one embodiment, the system maps XRP issued asset unfreezes to ISO 20022 seev.039 messages.
83. In one embodiment, the system maps XRP trustline freezes to ISO 20022 seev.040 messages.
84. In one embodiment, the system maps XRP trustline unfreezes to ISO 20022 seev.041 messages.
85. In one embodiment, the system maps XRP AMM pool creations to ISO 20022 seev.042 messages.
86. In one embodiment, the system maps XRP AMM pool deletions to ISO 20022 seev.043 messages.
87. In one embodiment, the system maps XRP hook-based RWA anchors to ISO 20022 colr.004 messages.
88. In one embodiment, the system maps XRP hook-based receipts to ISO 20022 seev.044 messages.
89. In one embodiment, the system maps XRP NFT royalties to ISO 20022 remt.003 messages.
90. In one embodiment, the system maps XRP NFT offers to ISO 20022 seev.045 messages.
91. In one embodiment, the system maps XRP NFT acceptances to ISO 20022 seev.046 messages.
92. In one embodiment, the system maps XRP NFT cancellations to ISO 20022 seev.047 messages.
93. In one embodiment, the system maps XRP AMM arbitrage events to ISO 20022 fxtr.005 messages.
94. In one embodiment, the system maps XRP AMM fee distributions to ISO 20022 camt.055 messages.
95. In one embodiment, the system maps XRP AMM pool rebalancing to ISO 20022 seev.048 messages.
96. In one embodiment, the system maps XRP hook-based covenant checks to ISO 20022 seev.049 messages.
97. In one embodiment, the system maps XRP hook-based oracle checks to ISO 20022 seev.050 messages.
98. In one embodiment, the system maps XRP hook-based compliance checks to ISO 20022 admi.006 messages.

99. In one embodiment, the system maps XRP hook-based routing decisions to ISO 20022 pacs.014 messages.
100. In one embodiment, the system maps XRP hook-based settlement instructions to ISO 20022 pacs.015 messages.

BLOCK 3 — EVM → ISO → ANY RAIL (101–150)

(Ethereum, Polygon, BSC, Avalanche, etc.)

101. In one embodiment, the system maps EVM ERC-20 transfers to ISO 20022 pacs.008 messages.
102. In one embodiment, the system maps EVM ERC-721 transfers to ISO 20022 seev.031 messages.
103. In one embodiment, the system maps EVM ERC-1155 transfers to ISO 20022 seev.032 messages.
104. In one embodiment, the system maps EVM smart contract calls to ISO 20022 remt.001 messages.
105. In one embodiment, the system maps EVM logs to ISO 20022 remt.002 structured messages.
106. In one embodiment, the system maps EVM gas fees to ISO 20022 camt.054 messages.
107. In one embodiment, the system maps EVM block confirmations to ISO 20022 acmt.022 messages.
108. In one embodiment, the system maps EVM validator rewards to ISO 20022 camt.053 messages.
109. In one embodiment, the system maps EVM staking deposits to ISO 20022 seev.033 messages.
110. In one embodiment, the system maps EVM staking withdrawals to ISO 20022 seev.034 messages.
111. In one embodiment, the system maps EVM slashing events to ISO 20022 admi.003 messages.
112. In one embodiment, the system maps EVM MEV bundles to ISO 20022 pacs.009 messages.
113. In one embodiment, the system maps EVM DEX swaps to ISO 20022 fxtr.001 messages.
114. In one embodiment, the system maps EVM liquidity pool deposits to ISO 20022 seev.035 messages.
115. In one embodiment, the system maps EVM liquidity pool withdrawals to ISO 20022 seev.036 messages.
116. In one embodiment, the system maps EVM LP token transfers to ISO 20022 seev.037 messages.
117. In one embodiment, the system maps EVM oracle updates to ISO 20022 fxtr.003 messages.

118. In one embodiment, the system maps EVM governance votes to ISO 20022 admi.007 messages.
119. In one embodiment, the system maps EVM governance proposals to ISO 20022 admi.008 messages.
120. In one embodiment, the system maps EVM governance executions to ISO 20022 admi.009 messages.
121. In one embodiment, the system maps EVM NFT mints to ISO 20022 seev.038 messages.
122. In one embodiment, the system maps EVM NFT burns to ISO 20022 seev.039 messages.
123. In one embodiment, the system maps EVM NFT metadata updates to ISO 20022 remt.003 messages.
124. In one embodiment, the system maps EVM cross-chain bridge events to ISO 20022 pacs.010 messages.
125. In one embodiment, the system maps EVM rollup deposits to ISO 20022 pacs.011 messages.
126. In one embodiment, the system maps EVM rollup withdrawals to ISO 20022 pacs.012 messages.
127. In one embodiment, the system maps EVM L2 batch proofs to ISO 20022 auth.006 messages.
128. In one embodiment, the system maps EVM L2 fraud proofs to ISO 20022 auth.007 messages.
129. In one embodiment, the system maps EVM L2 validity proofs to ISO 20022 auth.008 messages.
130. In one embodiment, the system maps EVM account abstraction operations to ISO 20022 auth.009 messages.
131. In one embodiment, the system maps EVM paymaster events to ISO 20022 camt.056 messages.
132. In one embodiment, the system maps EVM bundler operations to ISO 20022 pacs.013 messages.
133. In one embodiment, the system maps EVM gas sponsorship events to ISO 20022 camt.057 messages.
134. In one embodiment, the system maps EVM contract self-destruct events to ISO 20022 admi.010 messages.
135. In one embodiment, the system maps EVM contract creation events to ISO 20022 acmt.026 messages.
136. In one embodiment, the system maps EVM RWA token mints to ISO 20022 seev.040 messages.
137. In one embodiment, the system maps EVM RWA token burns to ISO 20022 seev.041 messages.
138. In one embodiment, the system maps EVM collateral locks to ISO 20022 colr.001 messages.
139. In one embodiment, the system maps EVM collateral releases to ISO 20022 colr.002 messages.
140. In one embodiment, the system maps EVM liquidation events to ISO 20022 colr.003 messages.

- 141. In one embodiment, the system maps EVM stablecoin mints to ISO 20022 pacs.014 messages.
- 142. In one embodiment, the system maps EVM stablecoin burns to ISO 20022 pacs.015 messages.
- 143. In one embodiment, the system maps EVM stablecoin transfers to ISO 20022 pacs.008 messages.
- 144. In one embodiment, the system maps EVM flash loans to ISO 20022 pacs.016 messages.
- 145. In one embodiment, the system maps EVM flash loan repayments to ISO 20022 pacs.017 messages.
- 146. In one embodiment, the system maps EVM liquidation auctions to ISO 20022 seev.042 messages.
- 147. In one embodiment, the system maps EVM oracle-based triggers to ISO 20022 seev.043 messages.
- 148. In one embodiment, the system maps EVM covenant-enforced assets to ISO 20022 seev.044 messages.
- 149. In one embodiment, the system maps EVM compliance-gated asset transfers to ISO 20022 admi.015 messages.

BLOCK 4 — EVM → ISO → ANY RAIL (continued, 151–175)

151. In one embodiment, the system maps EVM covenant-locked assets to ISO 20022 seev.045 messages.
152. In one embodiment, the system maps EVM synthetic asset mints to ISO 20022 seev.046 messages.
153. In one embodiment, the system maps EVM synthetic asset burns to ISO 20022 seev.047 messages.
154. In one embodiment, the system maps EVM synthetic asset transfers to ISO 20022 pacs.008 messages.
155. In one embodiment, the system maps EVM wrapped asset mints to ISO 20022 seev.048 messages.
156. In one embodiment, the system maps EVM wrapped asset burns to ISO 20022 seev.049 messages.
157. In one embodiment, the system maps EVM wrapped asset transfers to ISO 20022 pacs.009 messages.
158. In one embodiment, the system maps EVM cross-domain messages to ISO 20022 remt.004 messages.
159. In one embodiment, the system maps EVM bridge lock events to ISO 20022 colr.004 messages.
160. In one embodiment, the system maps EVM bridge mint events to ISO 20022 seev.050 messages.
161. In one embodiment, the system maps EVM bridge burn events to ISO 20022 seev.051 messages.
162. In one embodiment, the system maps EVM bridge unlock events to ISO 20022 colr.005 messages.
163. In one embodiment, the system maps EVM chain reorg events to ISO 20022 admi.011 messages.
164. In one embodiment, the system maps EVM chain finality checkpoints to ISO 20022 acmt.027 messages.
165. In one embodiment, the system maps EVM validator set changes to ISO 20022 auth.010 messages.
166. In one embodiment, the system maps EVM gas price updates to ISO 20022 fxtr.006 messages.
167. In one embodiment, the system maps EVM L2 sequencer batches to ISO 20022 pacs.018 messages.
168. In one embodiment, the system maps EVM L2 batch reverts to ISO 20022 admi.012 messages.
169. In one embodiment, the system maps EVM L2 state roots to ISO 20022 auth.011 messages.
170. In one embodiment, the system maps EVM L2 message receipts to ISO 20022 seev.052 messages.
171. In one embodiment, the system maps EVM cross-rollup transfers to ISO 20022 pacs.019 messages.
172. In one embodiment, the system maps EVM account abstraction paymaster approvals to ISO 20022 auth.012 messages.

173. In one embodiment, the system maps EVM account abstraction paymaster denials to ISO 20022 admi.013 messages.
174. In one embodiment, the system maps EVM zk-proof verifications to ISO 20022 auth.013 messages.
175. In one embodiment, the system maps EVM zk-proof failures to ISO 20022 admi.014 messages.

BLOCK 5 — STELLAR → ISO → ANY RAIL (176–200)

- 176. In one embodiment, the system maps Stellar payments to ISO 20022 pacs.008 messages.
- 177. In one embodiment, the system maps Stellar trustline creations to ISO 20022 seev.031 messages.
- 178. In one embodiment, the system maps Stellar trustline removals to ISO 20022 seev.032 messages.
- 179. In one embodiment, the system maps Stellar asset issuances to ISO 20022 seev.033 messages.
- 180. In one embodiment, the system maps Stellar asset burns to ISO 20022 seev.034 messages.
- 181. In one embodiment, the system maps Stellar DEX offers to ISO 20022 fxtr.001 messages.
- 182. In one embodiment, the system maps Stellar DEX trades to ISO 20022 fxtr.002 messages.
- 183. In one embodiment, the system maps Stellar account merges to ISO 20022 acmt.028 messages.
- 184. In one embodiment, the system maps Stellar inflation events to ISO 20022 camt.053 messages.
- 185. In one embodiment, the system maps Stellar claimable balances to ISO 20022 pacs.009 messages.
- 186. In one embodiment, the system maps Stellar sponsorship events to ISO 20022 remt.001 messages.
- 187. In one embodiment, the system maps Stellar clawback operations to ISO 20022 seev.035 messages.
- 188. In one embodiment, the system maps Stellar clawback claimable balances to ISO 20022 seev.036 messages.
- 189. In one embodiment, the system maps Stellar liquidity pool deposits to ISO 20022 seev.037 messages.
- 190. In one embodiment, the system maps Stellar liquidity pool withdrawals to ISO 20022 seev.038 messages.
- 191. In one embodiment, the system maps Stellar liquidity pool trades to ISO 20022 fxtr.003 messages.
- 192. In one embodiment, the system maps Stellar signer changes to ISO 20022 auth.001 messages.
- 193. In one embodiment, the system maps Stellar account flags to ISO 20022 acmt.029 messages.
- 194. In one embodiment, the system maps Stellar memo fields to ISO 20022 remt.002 messages.
- 195. In one embodiment, the system maps Stellar fee bumps to ISO 20022 camt.054 messages.
- 196. In one embodiment, the system maps Stellar sponsorship revocations to ISO 20022 admi.015 messages.
- 197. In one embodiment, the system maps Stellar claimable balance claims to ISO 20022 pacs.010 messages.

198. In one embodiment, the system maps Stellar liquidity pool share transfers to ISO 20022 seev.039 messages.
199. In one embodiment, the system maps Stellar AMM price updates to ISO 20022 fxtr.004 messages.
200. In one embodiment, the system maps Stellar RWA anchors to ISO 20022 colr.006 messages.

BLOCK 6 — BANK RAILS → ISO → ANY RAIL (201–225)

- 201. In one embodiment, the system maps ACH credits to ISO 20022 pacs.008 messages.
- 202. In one embodiment, the system maps ACH debits to ISO 20022 pacs.003 messages.
- 203. In one embodiment, the system maps ACH returns to ISO 20022 pacs.002 messages.
- 204. In one embodiment, the system maps ACH corrections to ISO 20022 camt.029 messages.
- 205. In one embodiment, the system maps SWIFT MT103 messages to ISO 20022 pacs.008 messages.
- 206. In one embodiment, the system maps SWIFT MT202 messages to ISO 20022 pacs.009 messages.
- 207. In one embodiment, the system maps SWIFT MT199 messages to ISO 20022 admi.002 messages.
- 208. In one embodiment, the system maps Fedwire transfers to ISO 20022 pacs.009 messages.
- 209. In one embodiment, the system maps SEPA credit transfers to ISO 20022 pain.001 messages.
- 210. In one embodiment, the system maps SEPA instant payments to ISO 20022 pacs.008 messages.
- 211. In one embodiment, the system maps SEPA direct debits to ISO 20022 pain.008 messages.
- 212. In one embodiment, the system maps CHAPS payments to ISO 20022 pacs.009 messages.
- 213. In one embodiment, the system maps RTGS settlements to ISO 20022 pacs.009 messages.
- 214. In one embodiment, the system maps bank ledger postings to ISO 20022 camt.054 messages.
- 215. In one embodiment, the system maps bank account openings to ISO 20022 acmt.001 messages.
- 216. In one embodiment, the system maps bank account closures to ISO 20022 acmt.002 messages.
- 217. In one embodiment, the system maps bank KYC updates to ISO 20022 auth.001 messages.
- 218. In one embodiment, the system maps bank AML flags to ISO 20022 admi.003 messages.
- 219. In one embodiment, the system maps bank loan disbursements to ISO 20022 camt.055 messages.
- 220. In one embodiment, the system maps bank loan repayments to ISO 20022 camt.056 messages.
- 221. In one embodiment, the system maps bank collateral pledges to ISO 20022 colr.001 messages.
- 222. In one embodiment, the system maps bank collateral releases to ISO 20022 colr.002 messages.

- 223. In one embodiment, the system maps bank FX trades to ISO 20022 fxtr.001 messages.
- 224. In one embodiment, the system maps bank securities trades to ISO 20022 seev.031 messages.
- 225. In one embodiment, the system maps bank settlement failures to ISO 20022 admi.004 messages.

BLOCK 7 — PRIVATE CHAINS / ENTERPRISE LEDGERS (226–250)

- 226. In one embodiment, the system maps Hyperledger Fabric chaincode events to ISO 20022 remt.001 messages.
- 227. In one embodiment, the system maps Hyperledger Fabric asset transfers to ISO 20022 pacs.008 messages.
- 228. In one embodiment, the system maps Hyperledger Fabric endorsement failures to ISO 20022 admi.003 messages.
- 229. In one embodiment, the system maps Hyperledger Fabric block commits to ISO 20022 acmt.022 messages.
- 230. In one embodiment, the system maps Hyperledger Besu smart contract calls to ISO 20022 remt.002 messages.
- 231. In one embodiment, the system maps Hyperledger Besu private transactions to ISO 20022 pacs.009 messages.
- 232. In one embodiment, the system maps Corda state transitions to ISO 20022 seev.031 messages.
- 233. In one embodiment, the system maps Corda notary signatures to ISO 20022 auth.002 messages.
- 234. In one embodiment, the system maps Corda flows to ISO 20022 remt.003 messages.
- 235. In one embodiment, the system maps Quorum private transactions to ISO 20022 pacs.010 messages.
- 236. In one embodiment, the system maps Quorum privacy group updates to ISO 20022 admi.005 messages.
- 237. In one embodiment, the system maps enterprise RWA token mints to ISO 20022 seev.040 messages.
- 238. In one embodiment, the system maps enterprise RWA token burns to ISO 20022 seev.041 messages.
- 239. In one embodiment, the system maps enterprise collateral pledges to ISO 20022 colr.001 messages.
- 240. In one embodiment, the system maps enterprise collateral releases to ISO 20022 colr.002 messages.
- 241. In one embodiment, the system maps enterprise supply chain events to ISO 20022 remt.004 messages.
- 242. In one embodiment, the system maps enterprise invoice tokenization to ISO 20022 remt.005 messages.
- 243. In one embodiment, the system maps enterprise payment approvals to ISO 20022 auth.003 messages.
- 244. In one embodiment, the system maps enterprise payment denials to ISO 20022 admi.006 messages.
- 245. In one embodiment, the system maps enterprise settlement batches to ISO 20022 pacs.011 messages.
- 246. In one embodiment, the system maps enterprise settlement failures to ISO 20022 admi.007 messages.
- 247. In one embodiment, the system maps enterprise identity proofs to ISO 20022 auth.004 messages.

- 248. In one embodiment, the system maps enterprise audit logs to ISO 20022 remt.006 messages.
- 249. In one embodiment, the system maps enterprise compliance flags to ISO 20022 admi.008 messages.
- 250. In one embodiment, the system maps enterprise covenant-locked assets to ISO 20022 seev.044 messages.

BLOCK 8 — CROSS-RAIL / MULTI-RAIL / META-EMBODIMENTS (251–300)

- 251. In one embodiment, the system routes Bitcoin → ISO → XRP.
- 252. In one embodiment, the system routes Bitcoin → ISO → Stellar.
- 253. In one embodiment, the system routes Bitcoin → ISO → EVM.
- 254. In one embodiment, the system routes Bitcoin → ISO → Bank rails.
- 255. In one embodiment, the system routes Bitcoin → ISO → Hyperledger.
- 256. In one embodiment, the system routes XRP → ISO → Bitcoin.
- 257. In one embodiment, the system routes XRP → ISO → Stellar.
- 258. In one embodiment, the system routes XRP → ISO → EVM.
- 259. In one embodiment, the system routes XRP → ISO → Bank rails.
- 260. In one embodiment, the system routes XRP → ISO → Hyperledger.
- 261. In one embodiment, the system routes Stellar → ISO → Bitcoin.
- 262. In one embodiment, the system routes Stellar → ISO → XRP.
- 263. In one embodiment, the system routes Stellar → ISO → EVM.
- 264. In one embodiment, the system routes Stellar → ISO → Bank rails.
- 265. In one embodiment, the system routes Stellar → ISO → Hyperledger.
- 266. In one embodiment, the system routes EVM → ISO → Bitcoin.
- 267. In one embodiment, the system routes EVM → ISO → XRP.
- 268. In one embodiment, the system routes EVM → ISO → Stellar.
- 269. In one embodiment, the system routes EVM → ISO → Bank rails.
- 270. In one embodiment, the system routes EVM → ISO → Hyperledger.
- 271. In one embodiment, the system routes Bank rails → ISO → Bitcoin.
- 272. In one embodiment, the system routes Bank rails → ISO → XRP.
- 273. In one embodiment, the system routes Bank rails → ISO → Stellar.
- 274. In one embodiment, the system routes Bank rails → ISO → EVM.
- 275. In one embodiment, the system routes Bank rails → ISO → Hyperledger.
- 276. In one embodiment, the system routes Hyperledger → ISO → Bitcoin.
- 277. In one embodiment, the system routes Hyperledger → ISO → XRP.
- 278. In one embodiment, the system routes Hyperledger → ISO → Stellar.
- 279. In one embodiment, the system routes Hyperledger → ISO → EVM.
- 280. In one embodiment, the system routes Hyperledger → ISO → Bank rails.
- 281. In one embodiment, the system performs multi-hop routing A → ISO → B → ISO → C.
- 282. In one embodiment, the system performs atomic multi-rail settlement.
- 283. In one embodiment, the system performs conditional multi-rail settlement.
- 284. In one embodiment, the system performs time-locked multi-rail settlement.
- 285. In one embodiment, the system performs oracle-locked multi-rail settlement.
- 286. In one embodiment, the system performs covenant-locked multi-rail settlement.
- 287. In one embodiment, the system performs RWA-anchored multi-rail settlement.
- 288. In one embodiment, the system performs FX-aware multi-rail routing.
- 289. In one embodiment, the system performs compliance-aware multi-rail routing.
- 290. In one embodiment, the system performs jurisdiction-aware multi-rail routing.
- 291. In one embodiment, the system performs asset-class-aware multi-rail routing.
- 292. In one embodiment, the system performs liquidity-aware multi-rail routing.
- 293. In one embodiment, the system performs fee-optimized multi-rail routing.

- 294. In one embodiment, the system performs risk-adjusted multi-rail routing.
- 295. In one embodiment, the system performs settlement-speed-optimized routing.
- 296. In one embodiment, the system performs redundancy-based multi-rail routing.
- 297. In one embodiment, the system performs failover multi-rail routing.
- 298. In one embodiment, the system performs audit-grade multi-rail lifecycle reconstruction.
- 299. In one embodiment, the system performs deterministic cross-rail replay.
- 300. In one embodiment, the system performs universal settlement across all supported rails using ISO 20022 as the canonical messaging layer.

INDEPENDENT CLAIM 1 — Universal Rail-to-ISO 20022 Interoperability System (Umbrella Claim)

1. **A computer-implemented system for enabling interoperability between heterogeneous financial rails**, the system comprising:
 - one or more inbound rail adapters configured to receive native transaction events from a plurality of financial rails and extract rail-specific transaction attributes including at least one of: sender identifier, receiver identifier, asset identifier, asset quantity, metadata, timestamp, and rail-specific state;
 - a canonical event model generator configured to transform the extracted transaction attributes into a rail-agnostic canonical representation comprising fields representing event_type, asset_type, asset_identity, amount, sender, receiver, metadata_hash, timestamp, rail_origin, rail_destination, and compliance_flags;
 - an ISO 20022 mapping engine configured to:
 - receive the canonical event model;
 - select an ISO 20022 message schema from a plurality of schemas including pacs.008, pain.001, seev.031, colr.001, fxtr.001, and camt.054 based on one or more fields of the canonical event model; and
 - map canonical event model fields to ISO 20022 message elements according to a predefined mapping table to generate an ISO 20022-compliant financial message;
 - a policy and compliance engine configured to evaluate the ISO 20022 message against jurisdictional rules, AML/KYC indicators, covenant constraints, and routing policies;
 - one or more outbound rail adapters configured to convert the ISO 20022 message into a native settlement instruction for a destination financial rail and submit the instruction to the destination rail;
 - a settlement coordinator configured to execute settlement across one or more destination rails, including performing atomicity, ordering, rollback, and multi-rail consistency operations; and
 - a lifecycle reconstruction engine configured to store canonical event logs, ISO 20022 message logs, and settlement confirmation logs, and to deterministically replay canonical transformations, ISO mappings, and settlement actions in chronological order.

DEPENDENT CLAIMS ON CLAIM 1 (1–5)

2. The system of claim 1, wherein the inbound rail adapters comprise protocol-specific extraction modules configured to retrieve transaction data using at least one of: RPC interfaces, WebSocket streams, REST APIs, message queues, or blockchain indexers.
3. The system of claim 1, wherein the canonical event model generator computes the metadata_hash using a cryptographic hash function selected from SHA-256, SHA-512, Keccak-256, or Blake2b.
4. The system of claim 1, wherein the ISO 20022 mapping engine selects the message schema using a rule-based schema selection algorithm that evaluates event_type, asset_class, settlement_mode, or compliance_flags.
5. The system of claim 1, wherein the policy and compliance engine applies jurisdictional rules based on sender location, receiver location, asset classification, or CBDC jurisdiction metadata.
6. The system of claim 1, wherein the outbound rail adapters generate native settlement instructions including at least one of: raw blockchain transactions, contract calls, trustline operations, ISO 20022 payment messages, or CBDC platform instructions.

INDEPENDENT CLAIM 2 — Canonical Event Conversion

7. A computer-implemented method for achieving interoperability between heterogeneous financial rails, the method comprising:
 - receiving, by an inbound rail adapter, a native transaction event from a first financial rail;
 - extracting rail-specific transaction attributes including at least one of: sender identifier, receiver identifier, asset identifier, asset quantity, metadata, timestamp, and rail-specific state;
 - transforming the extracted attributes into a Canonical Event Model (CEM) comprising rail-agnostic fields representing event_type, asset_type, asset_identity, amount, sender, receiver, metadata_hash, timestamp, rail_origin, rail_destination, and compliance_flags; and
 - storing the CEM in a machine-readable format for subsequent ISO 20022 mapping and cross-rail settlement.

DEPENDENT CLAIMS ON CLAIM 2 (8–12)

8. The method of claim 7, wherein the first financial rail comprises a Bitcoin network and the native transaction event comprises a UTXO-based spend including Taproot script paths or ordinal inscription metadata.
9. The method of claim 7, wherein the first financial rail comprises the XRP Ledger and the native transaction event comprises a payment, trustline modification, AMM operation, or issued asset transfer.
10. The method of claim 7, wherein the first financial rail comprises an EVM-compatible blockchain and the native transaction event comprises a contract log, ERC-20/721/1155 transfer, or smart contract state transition.
11. The method of claim 7, wherein the metadata_hash is computed from inscription data, covenant state, RWA metadata, or SST-10K structured metadata.
12. The method of claim 7, wherein compliance_flags include at least one of: jurisdiction restrictions, AML indicators, covenant constraints, or asset-class limitations.

INDEPENDENT CLAIM 3 — ISO 20022 Message Generation

13. A method for generating ISO 20022-compliant financial messages from canonical events, the method comprising:
- receiving a Canonical Event Model (CEM);
 - selecting an ISO 20022 message schema based on event_type or asset_class;
 - mapping CEM fields to ISO 20022 message elements according to a predefined mapping table;
 - generating a structured ISO 20022 message; and
 - outputting the ISO 20022 message for routing, compliance evaluation, or settlement.

DEPENDENT CLAIMS ON CLAIM 3 (14–18)

14. The method of claim 13, wherein mapping to pacs.008 is performed when the CEM event_type corresponds to a payment transfer.
15. The method of claim 13, wherein mapping to seev.031 is performed when the CEM asset_class corresponds to a securities instrument.
16. The method of claim 13, wherein mapping to colr.001 is performed when the CEM event_type corresponds to a collateral pledge or release.
17. The method of claim 13, wherein mapping to fxtr.001 is performed when the CEM event_type corresponds to a cross-asset or cross-currency exchange.
18. The method of claim 13, wherein the ISO 20022 message includes supplementary data containing the metadata_hash of the canonical event.

INDEPENDENT CLAIM 4 — Cross-Rail Settlement Execution

19. A method for executing settlement on a destination financial rail using ISO 20022 instructions, the method comprising:

- receiving an ISO 20022 message;
- selecting a destination rail adapter based on the rail_destination field;
- translating the ISO 20022 message into a native settlement instruction;
- submitting the native settlement instruction to the destination rail; and
- generating a settlement confirmation event.

DEPENDENT CLAIMS ON CLAIM 4 (20–22)

20. The method of claim 19, wherein the destination rail comprises a blockchain and the native settlement instruction comprises a signed transaction or contract invocation.
21. The method of claim 19, wherein the destination rail comprises a banking network and the native settlement instruction comprises an ISO 20022 pacs or pain message.
22. The method of claim 19, wherein settlement is atomic across multiple rails using a coordinated commit protocol, hash-time-locked contracts, or conditional settlement instructions.

INDEPENDENT CLAIM 5 — Rail Profile Object

23. A modular rail profile object enabling plug-and-play interoperability, the object comprising:

- a read_method;
- a write_method;
- an event_extractor;
- a signature_verifier;
- a settlement_method; and
- mapping_rules defining how rail-specific attributes map to canonical event fields and ISO 20022 elements.

DEPENDENT CLAIM ON CLAIM 5 (24)

24. The rail profile object of claim 23, wherein mapping_rules include reversible mappings enabling deterministic reconstruction of native events from canonical event model instances.

INDEPENDENT CLAIM 6 — Lifecycle Reconstruction Engine

25. A system for deterministic lifecycle reconstruction of cross-rail financial events, the system comprising:

- a canonical event log;
- an ISO 20022 log;
- a settlement log; and
- a replay engine configured to reconstruct the full lifecycle of any cross-rail event.

DEPENDENT CLAIM ON CLAIM 6 (26)

26. The system of claim 25, wherein the replay engine verifies cross-rail consistency by comparing canonical event hashes, ISO 20022 message hashes, and settlement confirmation hashes.

ASCII FIGURES

BLOCK DIAGRAM (ASCII)

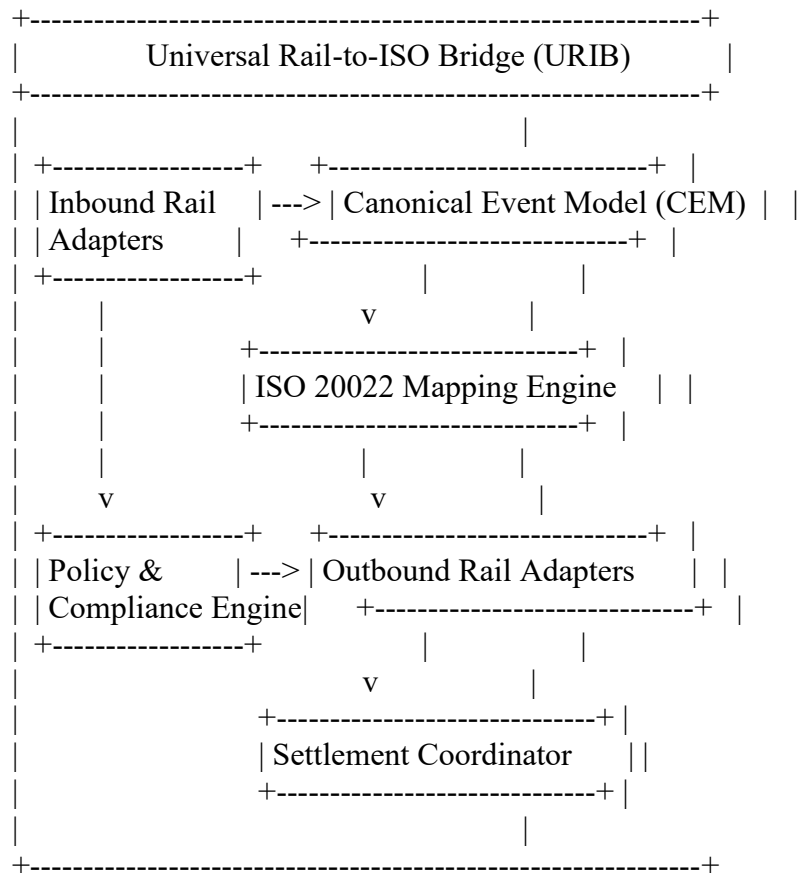
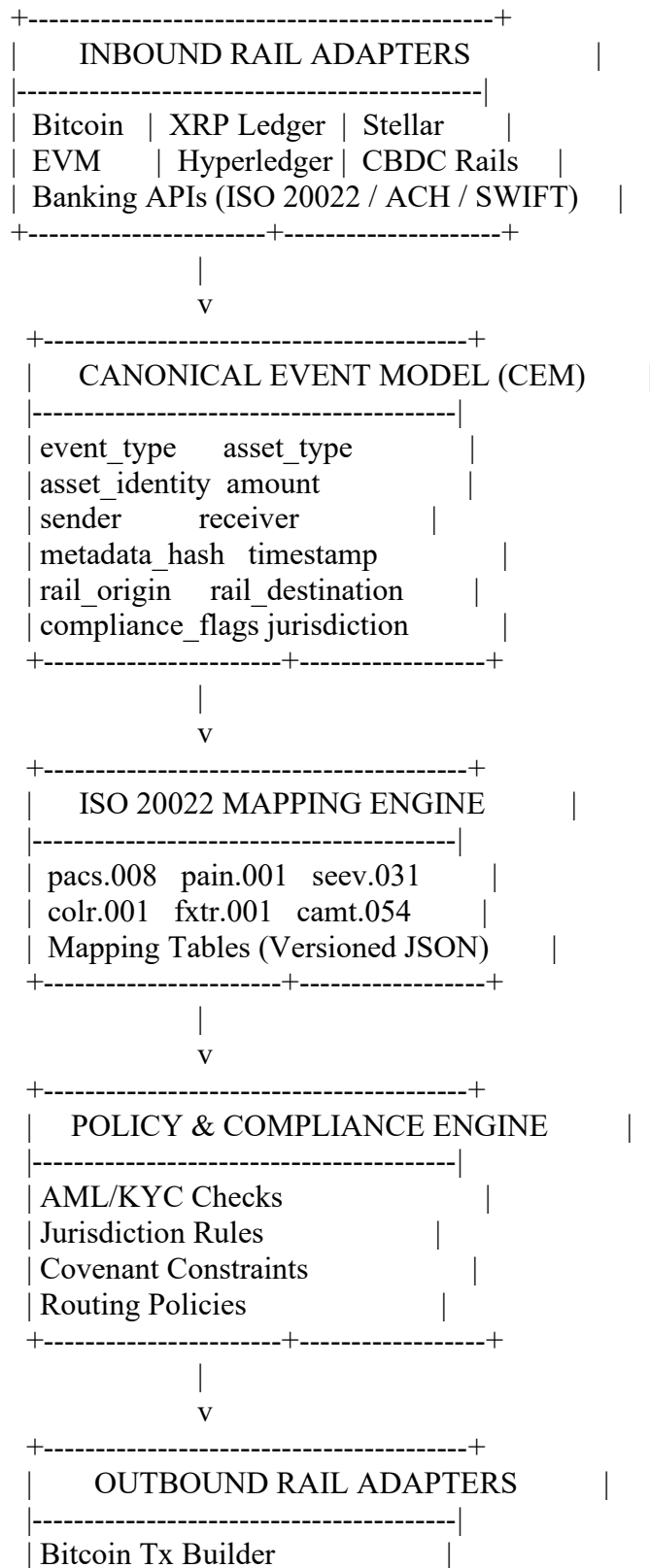


FIG. 1 — SYSTEM ARCHITECTURE OVERVIEW (ASCII BLOCK DIAGRAM)



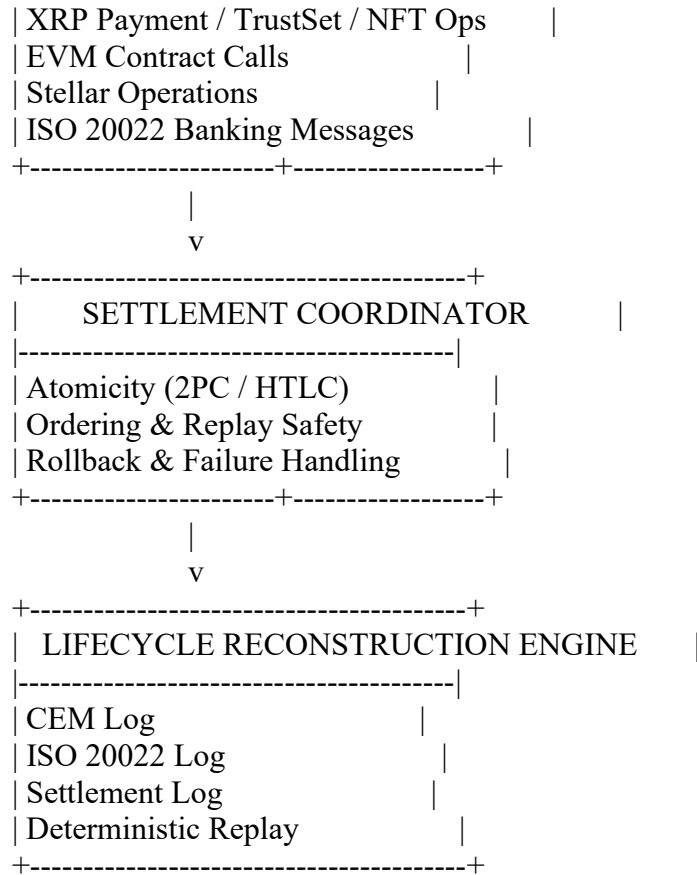


FIG. 2 — CANONICAL EVENT MODEL (CEM) STRUCTURE

(ASCII Data-Structure Diagram)

```

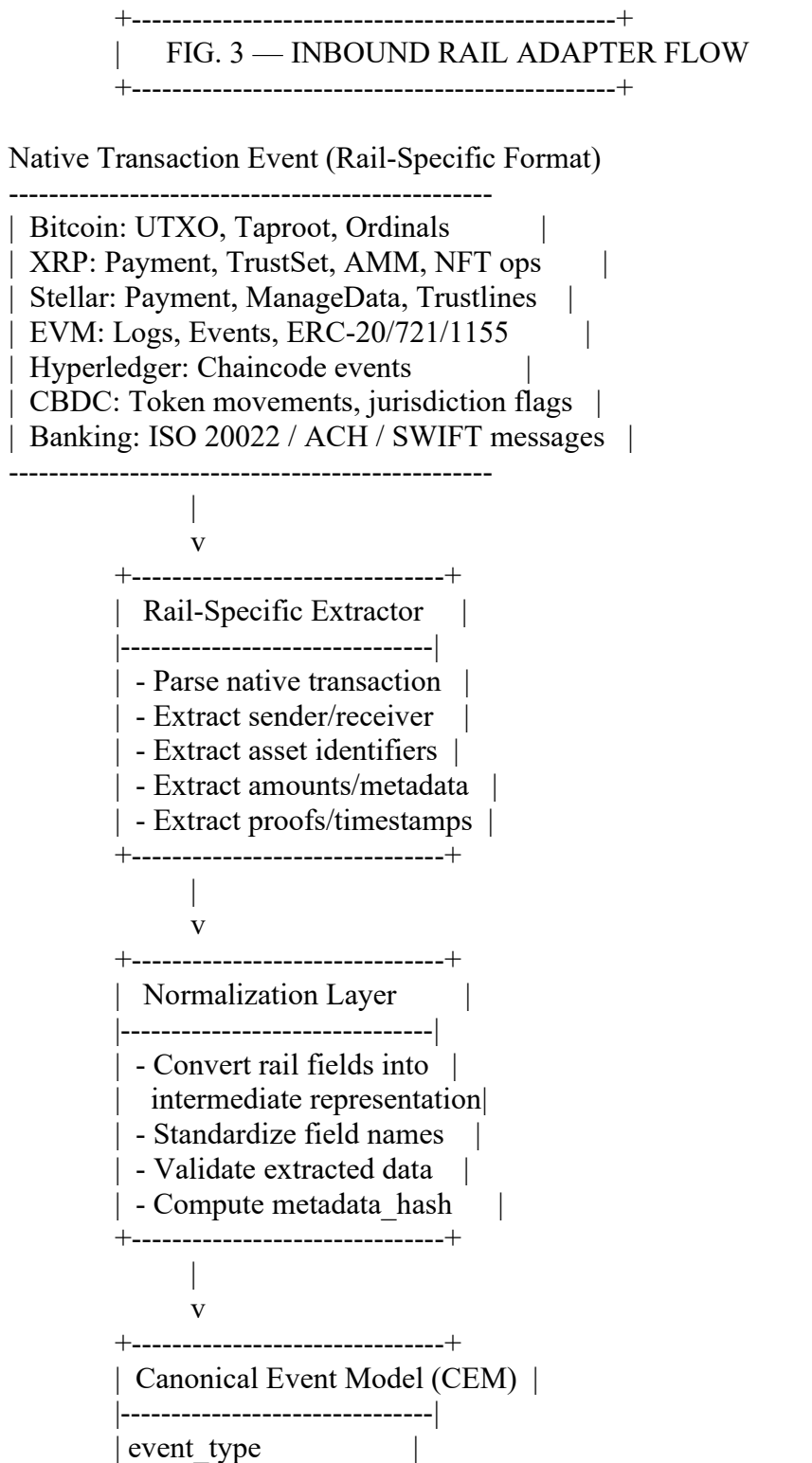
+-----+
|  FIG. 2 — CANONICAL EVENT MODEL (CEM)  |
+-----+
|  A rail-agnostic, deterministic data model  |
|  representing the essential semantics of  |
|  any financial event across all rails.  |
+-----+

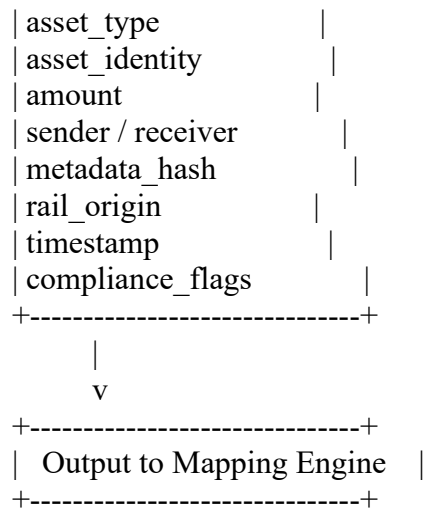
+-----+
|          CANONICAL EVENT MODEL          |
+-----+
| event_type      | transfer / mint / burn |
| asset_type      | BTC / XRP / ERC20 / RWA |
| asset_class     | payment / security  |
| asset_identity  | satoshi ID / token hash |
| amount         | numeric value  |
| unit           | satoshi / drops / wei |
| sender         | rail-specific address |
| receiver       | rail-specific address |
| metadata_hash   | SHA-256 / Keccak hash |
| metadata_uri    | optional URI  |
| rail_origin     | bitcoin / xrp / evm  |
| rail_destination | any supported rail  |
| compliance_flags | AML / jurisdiction / |
|                | covenant / asset rules |
| jurisdiction    | US / EU / JP / etc.  |
| covenant_state  | no-split / no-merge  |
| timestamp      | ledger timestamp  |
| expiry_time     | optional TTL  |
| proof_type      | merkle / zk / spv  |
| proof_data      | encoded proof bytes  |
| settlement_mode  | atomic / async  |
| settlement_priority | high / normal / low  |
+-----+

```

FIG. 3 — INBOUND RAIL ADAPTER FLOW

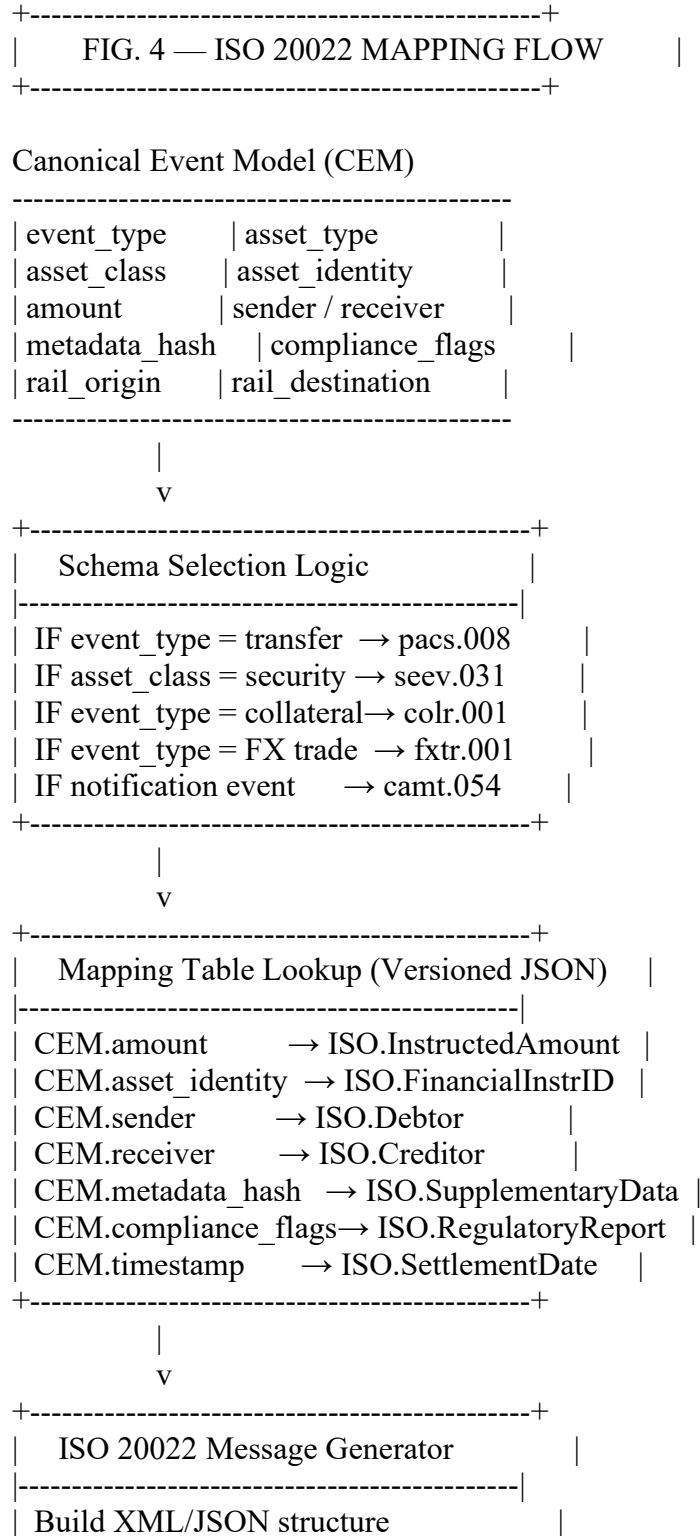
(ASCII Flow Diagram — Native Event → Extraction → Normalization → CEM)





ISO 20022 MAPPING FLOW

(ASCII Flow Diagram — CEM → Schema Selection → Mapping → ISO Message)



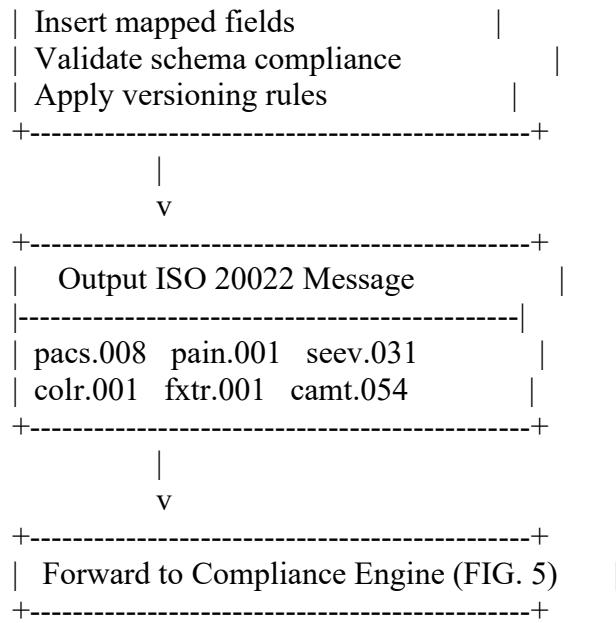
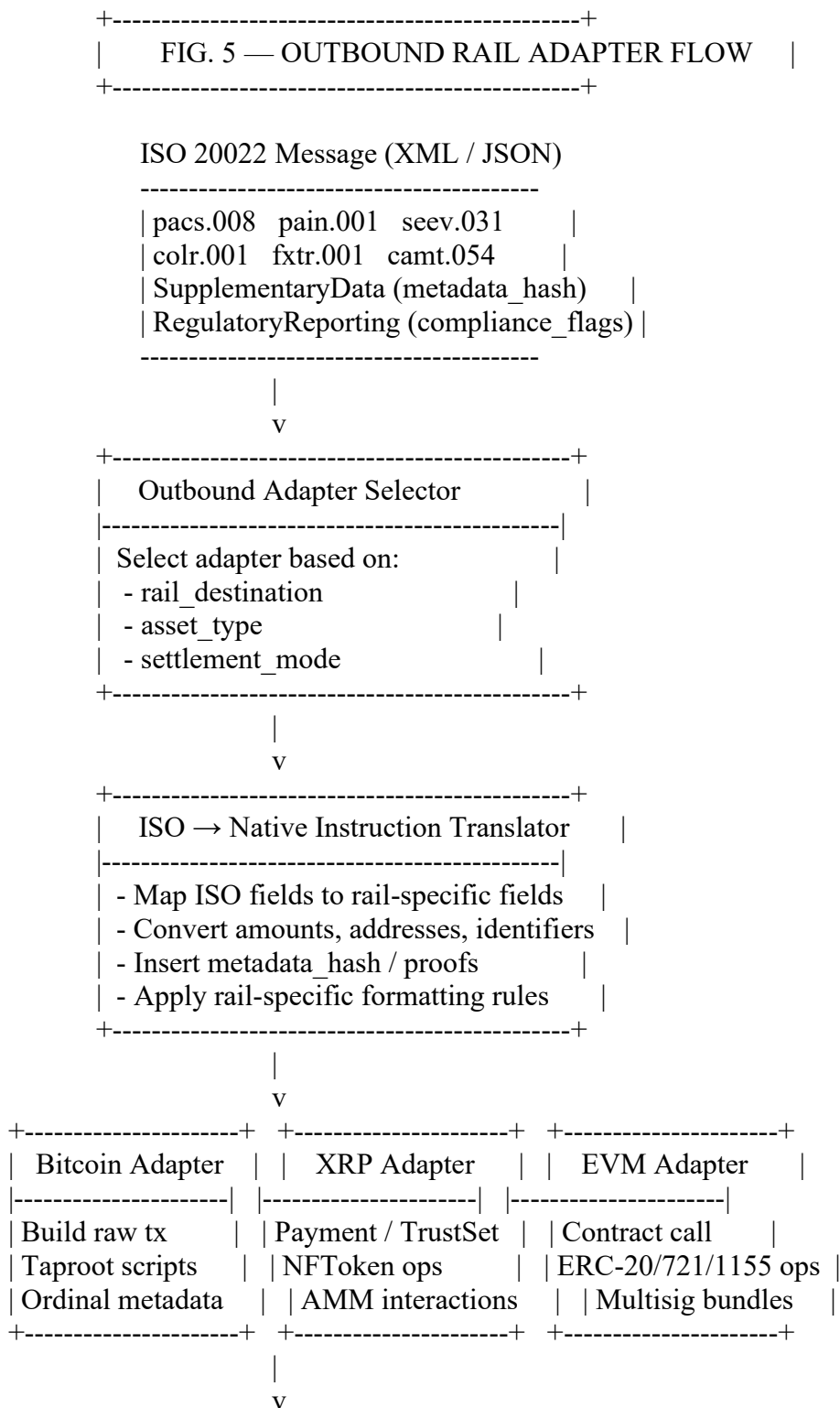


FIG. 5 — OUTBOUND RAIL ADAPTER FLOW

(ASCII Flow Diagram — ISO 20022 → Native Instruction → Destination Rail)



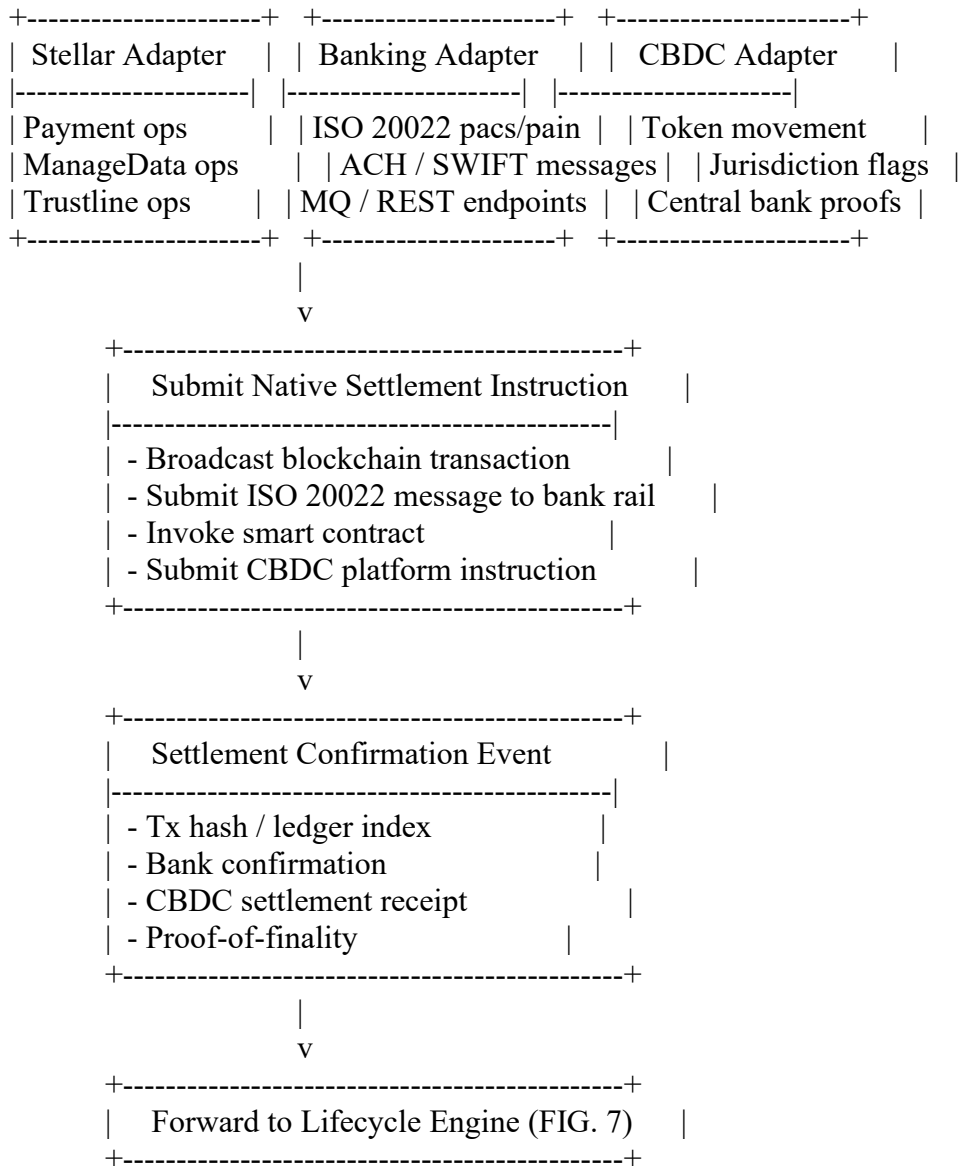
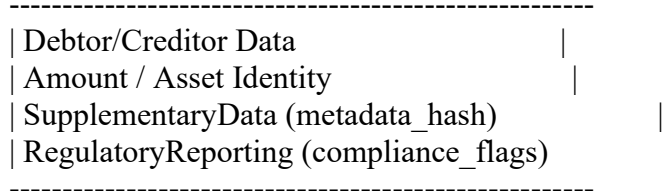


FIG. 6 — CROSS-RAIL SETTLEMENT SEQUENCE

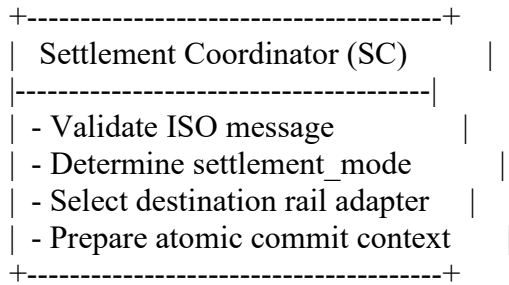
(ASCII Sequence Diagram — ISO → Destination Rail → Confirmation → Logs)



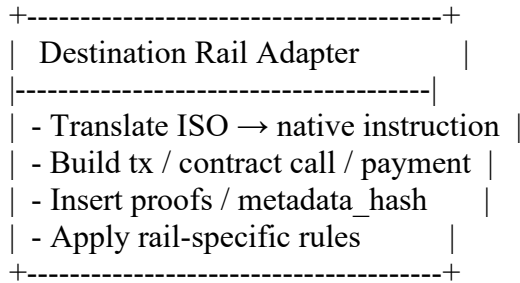
ISO 20022 Message
(pacs.008 / pain.001 / seev.031 / colr.001 / fxtr.001)



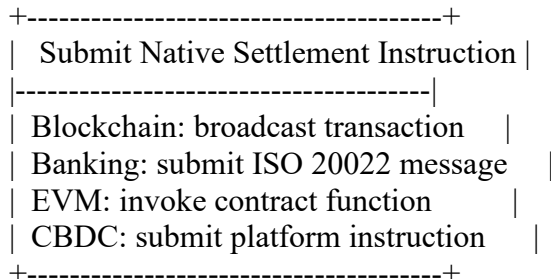
|
v



|
v



|
v



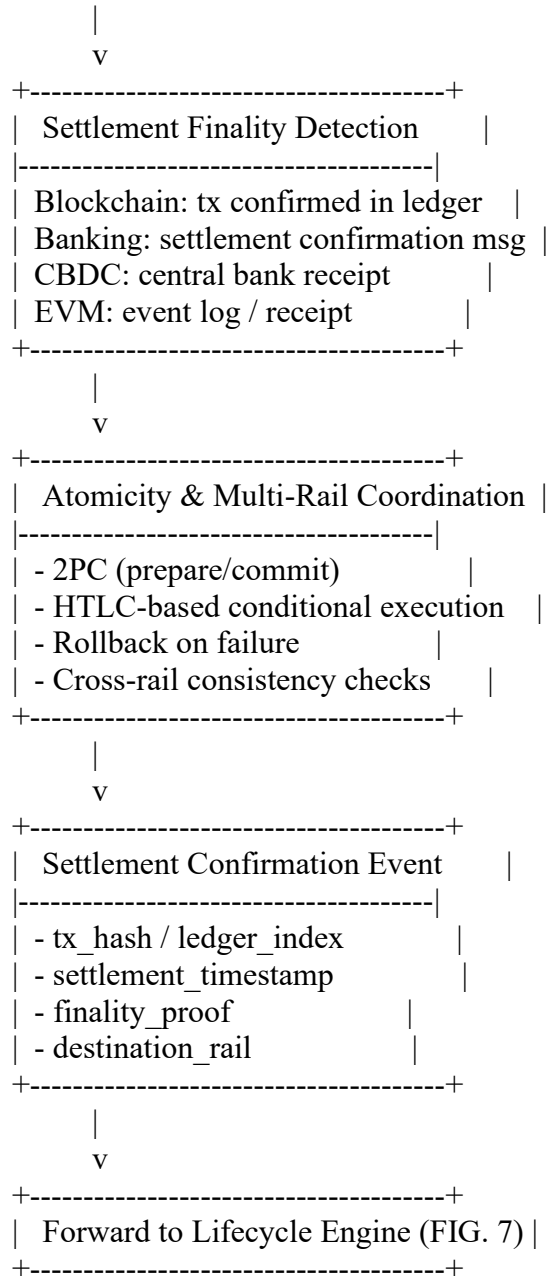
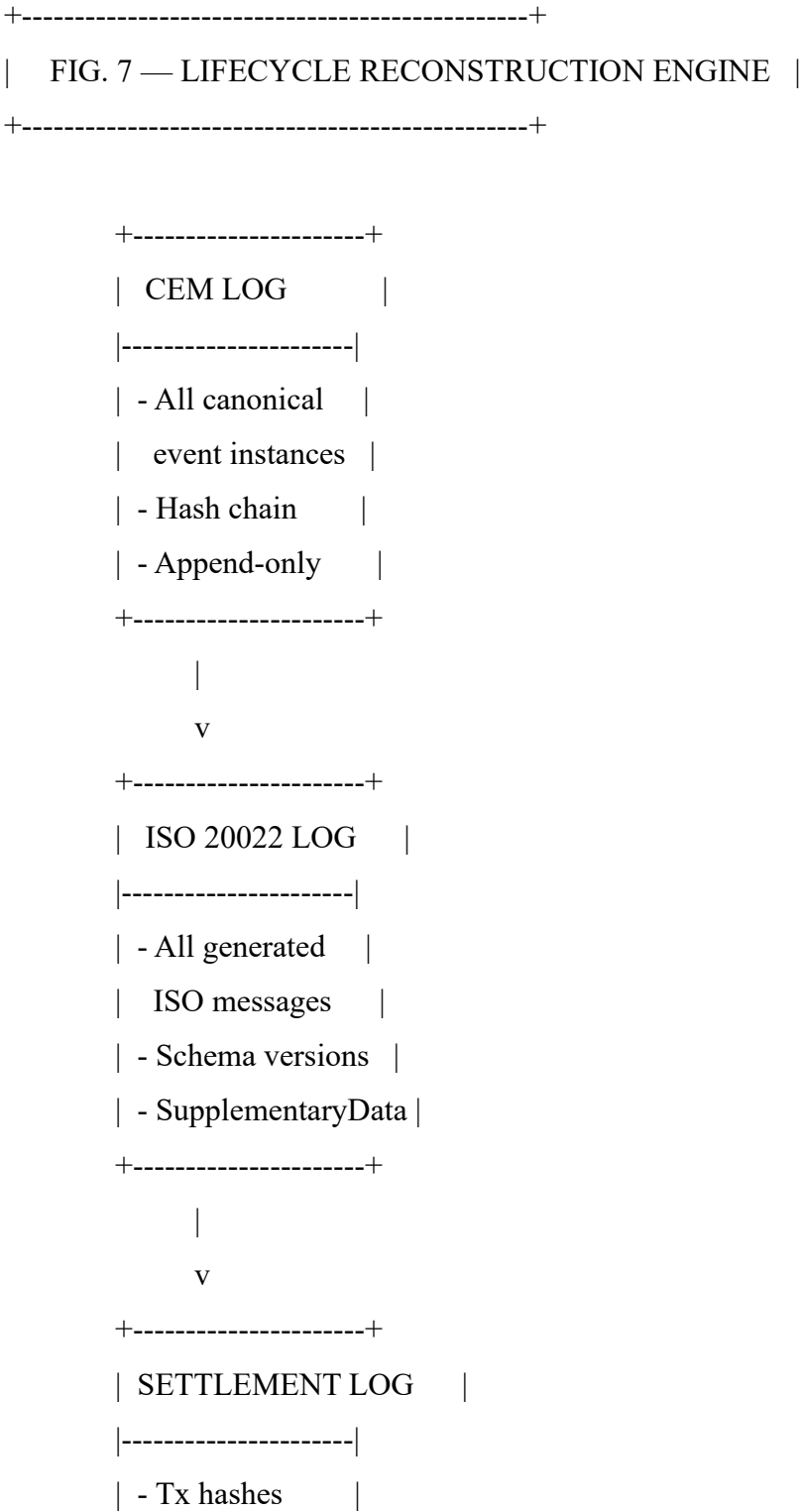
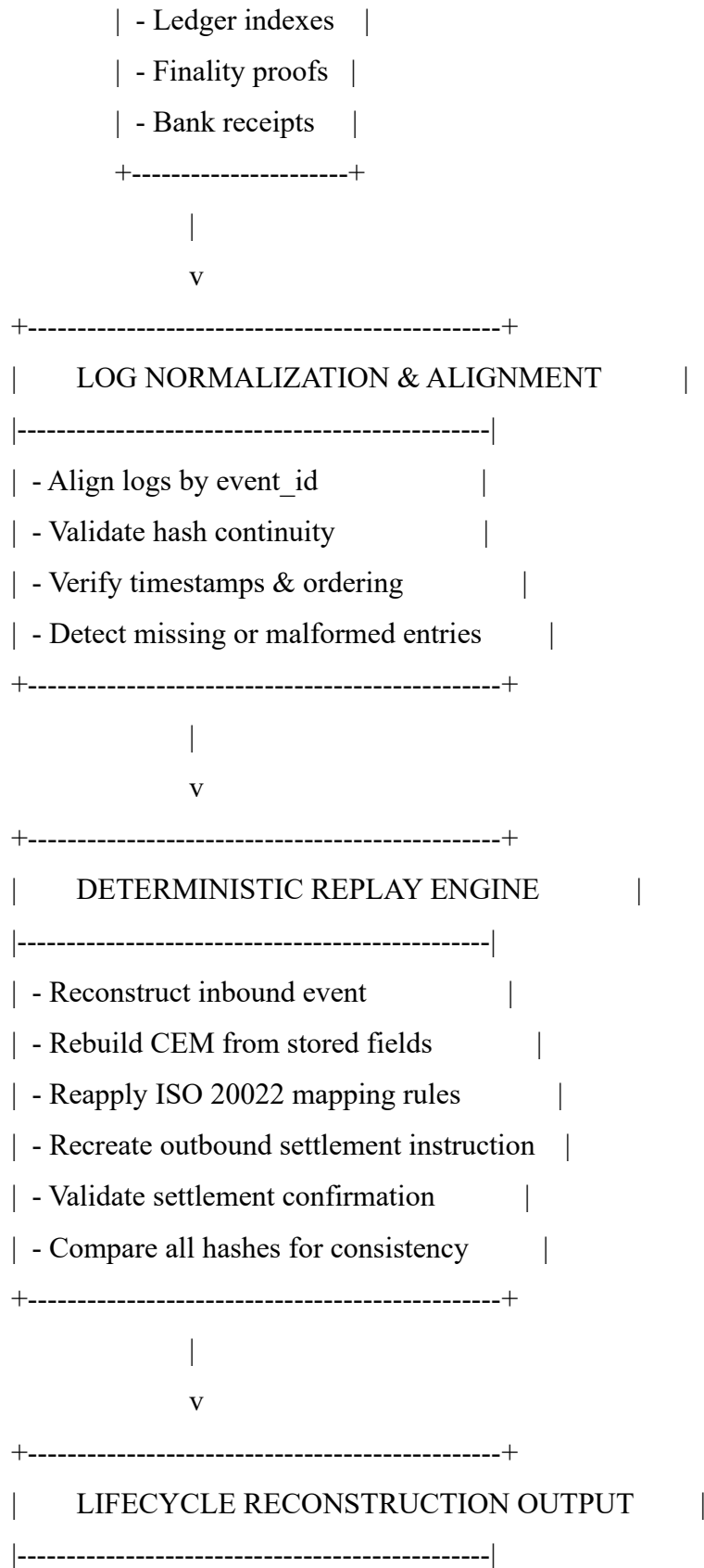


FIG. 7 — LIFECYCLE RECONSTRUCTION ENGINE

(ASCII Block Diagram — Logs → Replay Engine → Deterministic Lifecycle Output)

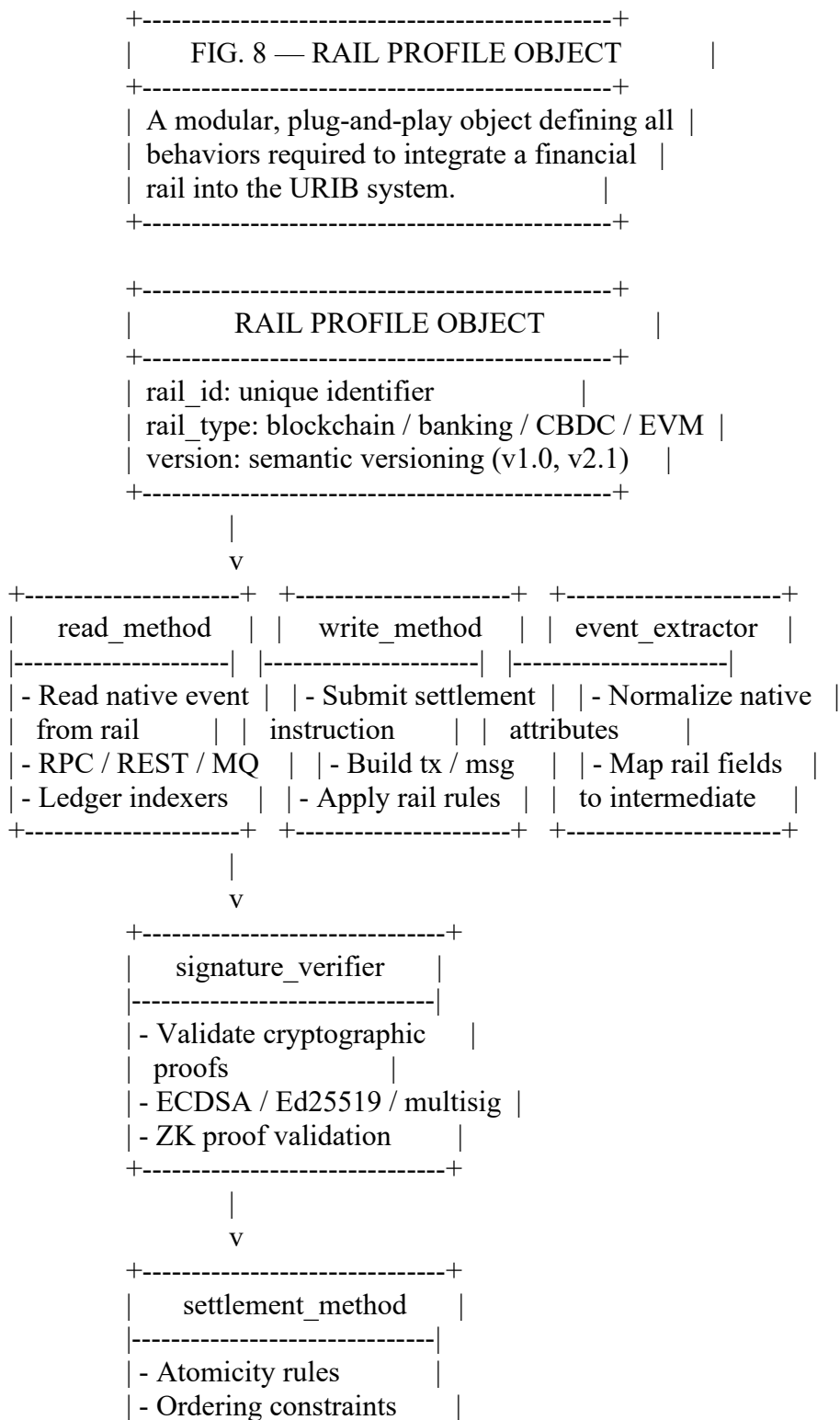




- | - Full event timeline |
 - | - Canonical → ISO → Settlement chain |
 - | - Compliance audit trail |
 - | - Deterministic replay proof |
- +-----+

RAIL PROFILE OBJECT STRUCTURE

(ASCII Class-Style Diagram — Methods + Mapping Rules + Verification)



- Finality detection	
- Rollback behavior	
+-----+	
	v
+-----+	
mapping_rules	
+-----+	
CEM.event_type	→ ISO.BusinessType
CEM.amount	→ ISO.InstructedAmount
CEM.asset_identity	→ ISO.FinInstrID
CEM.sender	→ ISO.Debtor
CEM.receiver	→ ISO.Creditor
CEM.metadata_hash	→ ISO.SupplementaryData
CEM.compliance_flags	→ ISO.RegulatoryReport
+-----+	
Reversible Mapping:	
ISO → CEM reconstruction for lifecycle replay	
+-----+	

FIG. 9 — MULTI-RAIL ROUTING EXAMPLE

(ASCII Flow Diagram — *Bitcoin* → *ISO* → *XRP* → *ISO* → *Banking Rail*)

```
+-----+
|   FIG. 9 — MULTI-RAIL ROUTING EXAMPLE   |
+-----+
```

STEP 1 — BITCOIN EVENT (SOURCE RAIL)

```
-----
| Native Bitcoin Tx (UTXO / Taproot / Ordinals) |
-----

      |
      v

+-----+
| Inbound Rail Adapter (Bitcoin) |
+-----+
| - Parse UTXO inputs/outputs    |
| - Extract sender/receiver      |
| - Extract satoshi amount       |
| - Extract inscription metadata |
+-----+

      |
      v

+-----+
| Canonical Event Model (CEM)    |
+-----+
| event_type: transfer           |
| asset_type: BTC                |
| asset_identity: satoshi_id / script |
| amount: 0.015 BTC              |
| sender: bc1...                 |
| receiver: xrp... (destination rail) |
+-----+
```

```

| metadata_hash: <hash>          |
| rail_origin: bitcoin            |
| rail_destination: xrp          |
+-----+
|
v

```

STEP 2 — ISO 20022 MESSAGE GENERATION

```

+-----+
| ISO 20022 Mapping Engine      |
|-----|
| - Select pacs.008            |
| - Map CEM fields → ISO fields |
| - Insert metadata_hash       |
+-----+
|
v
+-----+
| ISO 20022 Message (pacs.008) |
+-----+
|
v

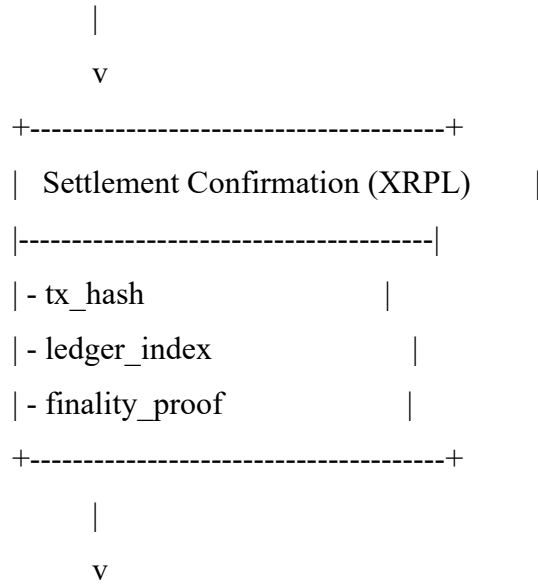
```

STEP 3 — XRP LEDGER SETTLEMENT (INTERMEDIATE RAIL)

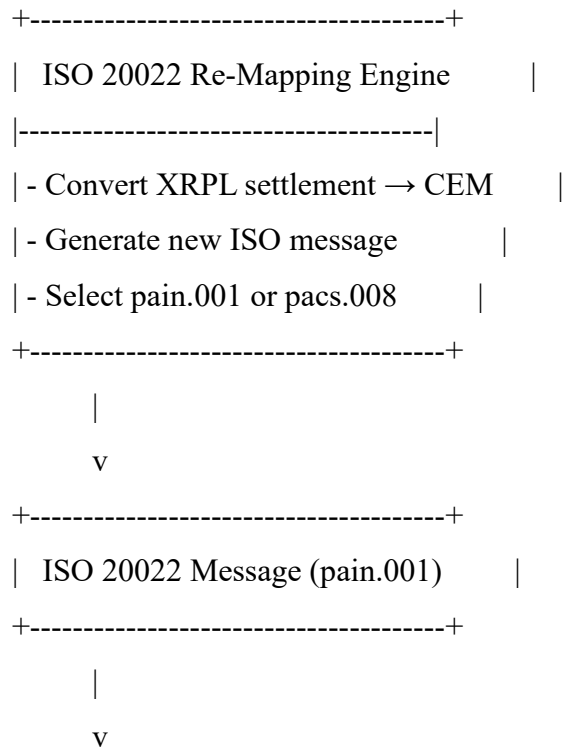
```

+-----+
| Outbound Rail Adapter (XRP)   |
|-----|
| - Build Payment / TrustSet / AMM op |
| - Insert metadata_hash       |
| - Submit to XRPL             |
+-----+

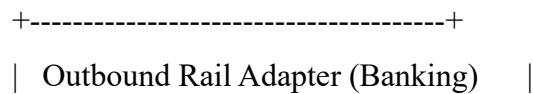
```

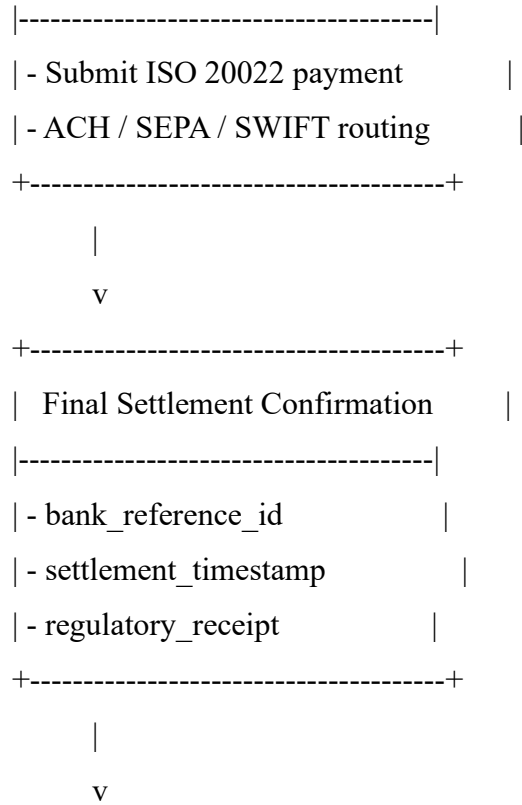


STEP 4 — ISO 20022 RE-ENCODING FOR BANKING RAIL



STEP 5 — BANKING RAIL SETTLEMENT (FINAL RAIL)





STEP 6 — LIFECYCLE RECONSTRUCTION ENGINE (FIG. 7)

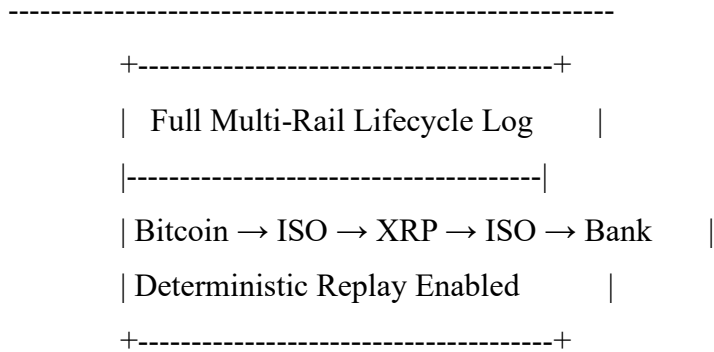


FIG. 10 — RWA / SST-10K INTEGRATION FLOW

(ASCII Flow Diagram — RWA Asset → CEM → ISO Securities/Collateral → Settlement → Replay)

```

+-----+
|   FIG. 10 — RWA / SST-10K INTEGRATION   |
+-----+

```

STEP 1 — RWA EVENT ON SOURCE RAIL

```

-----
| Native Event:                               |
| - Tokenized RWA (bond, note, invoice, property) |
| - SST-10K covenant-bound asset                |
| - Metadata: valuation, issuer, jurisdiction, rights |
| - Covenant rules: no-split, no-merge, transfer caps |
-----

```

|
v

```

+-----+
| Inbound Rail Adapter (Any Rail)             |
+-----+
| - Extract RWA metadata                       |
| - Extract covenant_state                     |
| - Extract issuer/owner identifiers           |
| - Extract asset_identity (hash/URI)          |
+-----+

```

|
v

STEP 2 — CANONICAL EVENT MODEL (CEM) FOR RWA

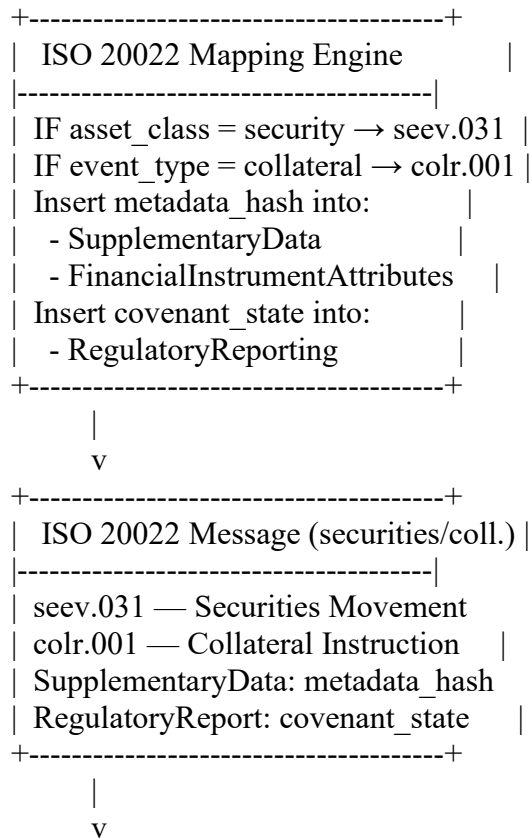
```

+-----+
| Canonical Event Model (CEM)                 |
+-----+
| event_type: transfer / pledge / issue        |
| asset_type: RWA                             |
| asset_class: security / collateral           |
| asset_identity: RWA_ID / SST10K_ID          |
| amount: face_value / units                  |
| metadata_hash: hash(RWA metadata)           |
| covenant_state: encoded SST-10K rules        |
| compliance_flags: jurisdiction, AML          |
| rail_origin / rail_destination              |
+-----+

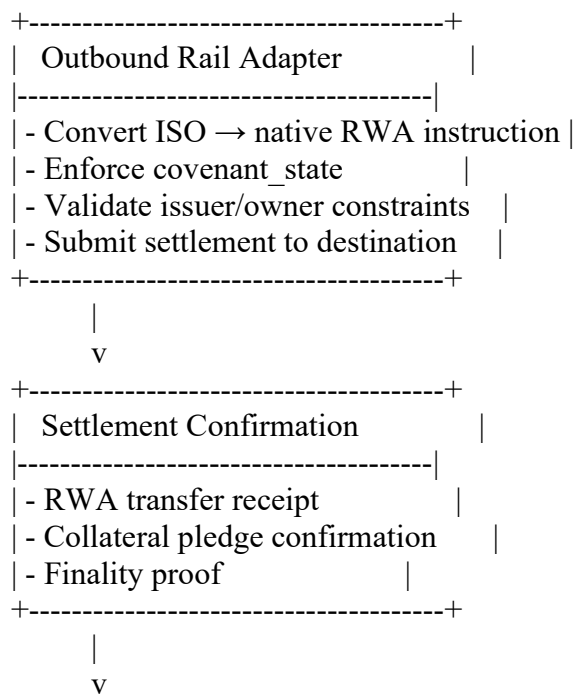
```

|
v

STEP 3 — ISO 20022 SECURITIES / COLLATERAL MAPPING



STEP 4 — OUTBOUND SETTLEMENT ON DESTINATION RAIL



STEP 5 — LIFECYCLE RECONSTRUCTION (FIG. 7)

